



The Open University

M263 Unit 5

UNDERGRADUATE ICT AND COMPUTING

Building Blocks of Software



The class Turtle

Unit 5



The Open University

M263 Building Blocks of Software

Unit 5

The class **Turtle**

Study guide

The first three sections of this unit are of about equal length and importance. Section 4 is “read only”. The work in Section 4 will not be directly assessed. It describes an interesting development of the material in the unit. It also provides a link to work (on recursion) in Block 4 of the course.

There are practical computing activities involving use of the WorkPad associated with this unit. These are given separately. We suggest that you spend about one-fifth of your study time for this unit on practical work.

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwenq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2004. Second edition 2008.

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd of 90 Tottenham Court Road, London W1T 4LP.

Edited, designed and typeset by The Open University, using the Open University T_EX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Wakefield.

ISBN 978 07492 2298 7

Contents

Introduction	4
1 Specifying the class Turtle	4
1.1 The data set	5
1.2 Applying the methods	9
1.3 From code to diagram	11
1.4 Marker state invariance	16
1.5 From diagram to code	17
2 Implementing the class Turtle	20
2.1 The attributes	21
2.2 The class Marker	21
2.3 Composition of classes	23
2.4 Using composition — an example	25
2.5 Implementing methods of Turtle	29
3 Extending the class Turtle	31
3.1 Rectangular patterns	31
3.2 Diagonal lines	36
4 Turtle fractals	40
Summary of Unit 5	45
Solutions to the exercises	46
Index	48

Introduction

In the late 1940s, Grey Walter, a neurophysiologist seeking to model brain function, carried out pioneering research on mobile autonomous robots at the Burden Neurological Institute. He conducted experiments with three-wheeled mobile robotic vehicles moving around on the floor in order to produce patterns. These tiny robots had a light sensor, touch sensor, motors for propulsion and steering, and a two-vacuum-tube analog computer. Even with this simple design, Walter showed that his robots exhibited complex behaviour. He called them *Machina Speculatrix* after their speculative tendency to explore their environment. His first robots were named Elmer and Elsie (ELeCtro MEchanical Robots, Light Sensitive). The tradition of calling such robots “turtles” started with this work. It in turn inspired the first MIT “turtle” robots, which responded to commands to drive them forward or to turn left or right.

In this unit, we shall describe a class **Turtle** whose objects are able to draw certain figures in two dimensions. In doing this, we shall explore further some general issues concerning the implementation of classes. The use of the name **Turtle** for this new class is no accident. It is an M263 version of the sort of commands available in the programming language LOGO.

FOLDOC, a computing dictionary, describes LOGO as a: *language for teaching programming, noted for its “turtle graphics” used to draw geometric shapes. LOGO was developed in 1966–1968 by a group at Bolt, Beranek and Newman headed by Wally Fuerzeig and including Seymour Papert.*

The commands of LOGO are quite easy to understand. The commands shown in the margin would cause a LOGO turtle to trace out a square. It does so by moving forward ten units of distance, then turning 90 degrees anticlockwise (to the left of the direction in which the turtle is heading). These two steps are repeated four times. The lines are plotted by a pen. Whether this pen is on the paper is part of the state of the turtle. When the pen is on, and the turtle is given the command FORWARD, it will move a specified distance in its current direction, marking a straight line as it moves. The pen can be taken off if you want the turtle to move without marking a line. The command TURN enables the turtle to change direction.

The language and its use in the classroom was further developed by the LOGO Group of the MIT Artificial Intelligence Laboratory in the 1970s, led by Seymour Papert. Geometric patterns could either be traced out on paper by small machines or shown on a computer screen. The lines and figures drawn are often referred to as the figures of Turtle Geometry. The *Lego_{TM}* robots that can be built from a *MindStorm_{TM}* kit can be programmed in LOGO, so that it is possible to build an elementary turtle robot.

For further details, see Walter, W. G., “An Imitation of Life”, *Scientific American*, May 1950, pp. 42–45, or Ward, M., “Walter’s world”, *New Scientist*, July 1998, p. 54.

See <http://wombat.doc.ic.ac.uk/foldoc>

TO SQUARE

```
FORWARD 10
LEFT 90
FORWARD 10
LEFT 90
FORWARD 10
LEFT 90
FORWARD 10
LEFT 90
```

An excellent introduction to the subject is to be found in “Turtle Geometry” by Abelson and DiSessa, MIT press, 1982.

1 Specifying the class Turtle

In LOGO, a series of commands can be given that will cause the turtle to trace out a square. On a computer screen, LOGO will then display the square as a diagram such as is shown in Figure 1.1. The small triangle at

the bottom left of Figure 1.1 represents the LOGO “turtle”. This moves around the square and in doing so marks out the lines. The triangle in the figure shows where the turtle currently is. As well as the position of the turtle it also shows the direction of the turtle, which is facing along the positive x -axis (and ready to move forward if given further commands).

The class **Spot** in Block 1 also has a graphical representation of its objects in the plane. An object of type **Spot** is shown as a single dot in the plane. A spot has methods that enable it to move in certain directions in the plane. A spot cannot make marks to draw a diagram, but we can use this available class to help in implementing a form of turtle. A line will be indicated by showing a number of consecutive spots. So a square drawn by an M263 turtle will look something like that shown in Figure 1.2.

Each “line” in Figure 1.2 consists of a number of separate spots. The “turtle” is again shown to have a position and a direction.

In this section we shall discuss the specification of the M263 class **Turtle**. We look first at the data set of this class, that is, the possible states that a turtle can have. We then look at the methods of **Turtle**, and how these can be used to draw certain figures.

1.1 The data set

The state of an object of type **Turtle** is determined by the state of four features, which are given below and explained more fully in the following text.

1. The marker.
2. The pen.
3. The diagram.
4. The markerStack.

The state of the marker consists of a position and a direction.

The state of the pen is either “on” or “off”.

The diagram consists of the points that have been marked.

The markerStack is a sequence of markers.

The data set of **Turtle** has a structure that is a good deal more complicated than any of the classes in Unit 4. We shall start with an informal discussion of this and then go on to specify it in more detail. For brevity, we shall shorten “an object of type **Turtle**” to “a turtle” from now on. The purpose of a turtle is to draw a diagram, so this discussion will be in graphical terms. Each of the four main features of the data set of **Turtle** is represented as an attribute in the class implementation. It is convenient to discuss the data set in terms of these attributes.

A turtle moves around locations on a two-dimensional grid, as did the spot objects you met in Unit 4. As well as a current position, a turtle has a current direction (the direction in which it can move). The turtle can move several places in its current direction. (This is achieved using a method *forward*.) If the pen is “on” then the turtle leaves behind it a trail of spots that are added to the diagram; if the pen is “off” then the turtle marks nothing.

So part of the state of the turtle is “where it is” and “the direction in which it can move”. To describe these features we use an attribute that we call the **marker**. This shows the current position of the turtle and the direction in which it is facing. The position of the marker is at a point on a two-dimensional grid. The marker has only four possible directions. It must

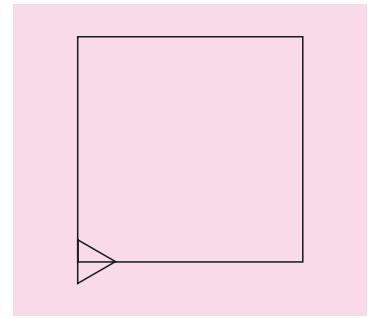


Figure 1.1

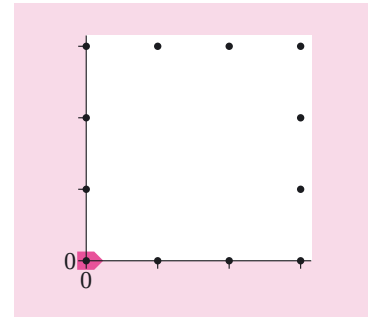


Figure 1.2

point along either the x -axis or the y -axis. Figure 1.3 shows a graphical representation of the marker feature of a turtle. It shows it at the position (2,3) and pointing to the right. The four possible marker directions are illustrated in Figure 1.4 (for a marker at the position (0,0)). We shall refer to these possible marker directions as follows: “right(0)”; “up(90)”; “left(180)” and “down(270)”. (The numbers here give the number of degrees you need to turn anticlockwise from the positive x -axis to get to each direction.)

The default value for the marker attribute of a turtle is (0,0) with direction “right(0)”. Directly after a **var** declaration a turtle has the state shown in Figure 1.4(a). The class **Turtle** has two methods that can change the direction of the marker. The method *leftTurn* will change the direction by 90 degrees anticlockwise. So applying *leftTurn* to the state shown in Figure 1.4(a) will change the state to that shown in Figure 1.4(b). The method *rightTurn* changes the direction by 90 degrees clockwise. So applying *rightTurn* to the state shown in Figure 1.4(a) will change the state to that shown in Figure 1.4(d). If *rightTurn* is applied twice to the state shown in Figure 1.4(a), then the state will change to that shown in Figure 1.4(c).

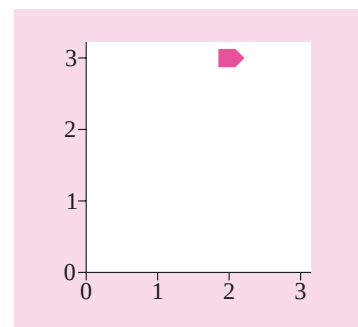


Figure 1.3

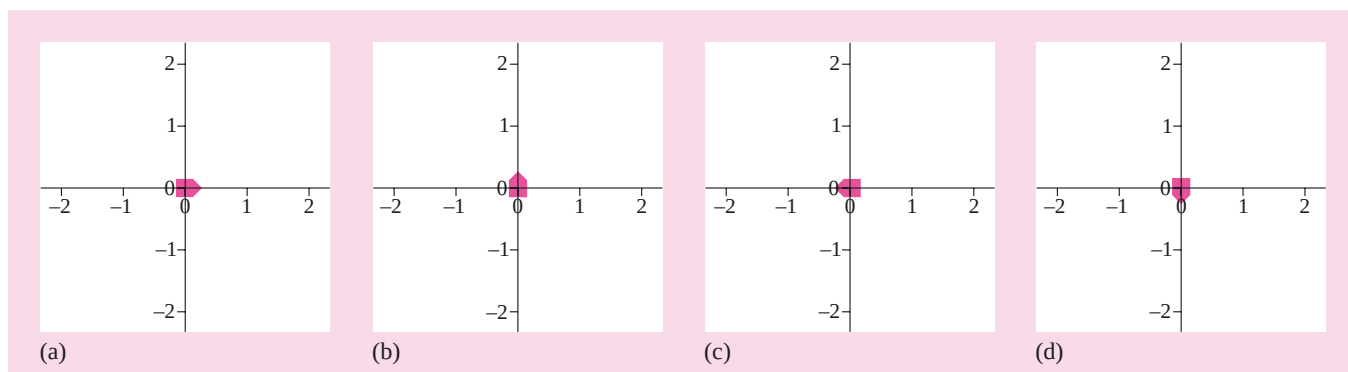


Figure 1.4 The four possible marker directions, for a marker at (0,0). (a) Marker direction “right(0)”. (b) Marker direction “up(90)”. (c) Marker direction “left(180)”. (d) Marker direction “down(270)”.

A turtle has a second attribute, called the **pen**, which can be in one of two states. If the pen is in the state “off” then no spots are left behind as the turtle moves. If the pen is “on” and the turtle is moved (using *forward*), then it moves a specified number of steps in the direction it is currently facing, and leaves a path of spots behind. Each of the spots on the path is then recorded as part of the overall state of the turtle.

The turtle’s third attribute is one that records the spots that have been marked. We will refer to this as the **diagram**. For example, consider the following code fragment.

```
var myTurtle in Turtle
myTurtle.forward(3)
```

The first statement declares an object *myTurtle*, which will initially be in the default state shown in Figure 1.4(a). The statement *myTurtle.forward(3)* will then move *myTurtle* three places in its current direction (“right(0)”). So it is moved three places along the positive x -axis, and will end at (3,0). In the default state, the pen is “on”. So as *myTurtle* moves in response to *forward*, it will leave a trail of spots behind. Each place on the grid that it has visited is marked, so this code will add to the diagram each integer

point on the x -axis from $(0,0)$ to $(3,0)$. The state of *myTurtle* after this code fragment is executed can be represented graphically, as in Figure 1.5.

The fourth attribute of a turtle, called the **markerStack**, makes it easier to write code to draw complicated diagrams. As the turtle's marker is moved and turned, we are able to store the marker's state at any given stage. Then at a later stage we can restore the marker to this stored state. (Further details of this are given later.)

When tracing code involving a turtle, we can usually follow its effect by drawing figures such as that in Figure 1.5. However, such a figure does not contain a full description of the state of a turtle. It does not show the state of the pen, nor does it show the state of the markerStack. These could be added as comments at the bottom of the figure if needed. We will write the markerStack as a sequence of marker states.

To show the state of a marker, we must give its position and its direction. The position can be given as a pair of coordinates, such as $(3,0)$. To give the marker direction, we will use “right(0)”, “up(90)”, “left(180)” and “down(270)”. For example, in Figure 1.5 the marker is in the state $(3,0, \text{“right(0)”})$.

In principle, we can show the entire state of a turtle in this sort of way. We would need to show the state of each of the four attributes. The marker state can be shown as above; the pen state will be either “on” or “off”; the markerStack will be a sequence of marker states. If we do this, we will show the diagram as a sequence. This sequence gives the spots in the diagram in the order in which they were marked. (This is how the WorkPad will record the spots in a turtle diagram.) For example, the diagram for Figure 1.5 is $[(0,0), (1,0), (2,0), (3,0)]$. However, we will usually show the diagram graphically rather than in this way.

In the default state of a turtle: the marker is at the origin $(0,0)$, with direction “right(0)”; the pen is “on”; the diagram contain no spots; and the markerStack is also empty (an empty sequence of marker states). We can write the default state as below:

```
((0,0,“right(0)”),“on”,[],[])
```

(Note that the first of the four attributes here is the default marker state, written as $(0,0, \text{“right(0)”})$.)

The four attributes fully describe the state of a turtle. The graphical representation of a turtle consists of a figure that shows the turtle's diagram and marker. Much of what we have said so far is summarised in the class specification overleaf.

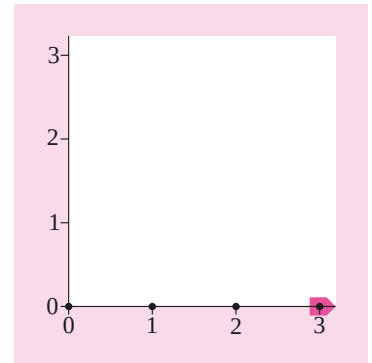


Figure 1.5

Turtle	Class specification
<i>States</i> An object of type Turtle (a turtle) has a state consisting of the following four attributes.	
• A marker, which itself has the following features. The position of the marker (a location on the grid, (x, y)). The direction of the marker (given as “right(0)”, “up(90)”, “left(180)” or “down(270)”). • A pen, which is either “on” or “off”. • A diagram, which consists of all the grid points that the turtle has passed over when its pen state is “on”. (These are stored in a sequence in the order that they were recorded.) • A markerStack, which is a sequence of markers, each being the state of the turtle’s marker at some point in its existence. (At a later stage, the turtle can have its marker state restored to the state of one of the stored markers.)	
<i>State notation</i> A 4-tuple giving the states of: the marker, the pen, the diagram and the markerStack.	
<i>Default</i> The marker is at the origin, with marker direction “right(0)”. The pen state is “on”. The diagram is empty. The markerStack is empty. So the default state is given by the 4-tuple $((0, 0, \text{“right(0)”}, \text{“on”}, [], [])$.	
<i>Methods</i> <i>forward</i>(<i>number in Int</i>) state change <i>leftTurn</i>() state change <i>rightTurn</i>() state change <i>penOff</i>() state change <i>penOn</i>() state change <i>store</i>() state change <i>restore</i>() state change	

When the pen state is “on”, the grid points passed over by the marker when moving *forward* are added to the diagram. When the pen state is “off”, no record is kept of the grid points as the turtle moves.

As with all M263 classes, **Turtle** extends **Base**, and so has access to *clone* and *show* in the WorkPad.

We now turn our attention to the methods that can be applied to a turtle. We give below a compact summary of the methods for the class. All the methods of the class **Turtle** change the state of the receiver object (none returns a value). The only method that requires a precondition other than **true** is *restore*, which requires that the markerStack is not empty.

<i>forward(number)</i>	Changes the state of the turtle as follows. If <i>number</i> = 0, the marker state does not change. If the pen is “on” then the current position of the marker is added to the diagram. Hence this single spot will be plotted even though the marker is not moved. If <i>number</i> > 0, the marker has its position changed by moving it <i>number</i> places in the current direction. If the pen is “on” then the initial marker position is stored in the diagram, as is each grid point that the marker passes over as it moves forward, including the final marker position. If <i>number</i> < 0, the turtle state is not changed.
<i>leftTurn()</i>	Changes the state of the turtle as follows. The direction of the marker is changed by turning 90 degrees in an anticlockwise direction. (See Figure 1.6.)
<i>rightTurn()</i>	Changes the state of the turtle as follows. The direction of the marker is changed by turning 90 degrees in a clockwise direction. (See Figure 1.7.)
<i>penOn()</i>	Changes the state of the turtle as follows. The pen state is set to “on”.
<i>penOff()</i>	Changes the state of the turtle as follows. The pen state is set to “off”.
<i>store()</i>	Changes the state of the turtle as follows. The current marker state of the turtle is added to the markerStack.
<i>restore()</i>	Changes the state of the turtle as follows. The turtle’s marker is set to the most recent marker state in the markerStack, and this state is then deleted from the markerStack.

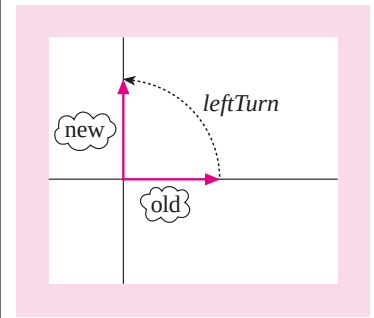


Figure 1.6 The effect of *leftTurn* on the marker direction.

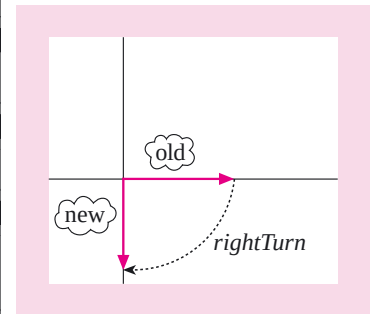


Figure 1.7 The effect of *rightTurn* on the marker direction.

Note that once a spot has been added to the diagram it cannot be erased. (There is no method that can delete a spot from the diagram.) The marker (with the pen “on”) may go *forward* over a spot that is already marked. In such a case, the sequence representing the diagram will contain more than one copy of that spot, but on a graphical representation of the turtle the spot will appear just once.

The class **Turtle** has no literals. A turtle can only be created in the default state, and can not be set to some other specified state. We can use *clone* to change the state of a turtle to become identical to the state of another turtle object.

1.2 Applying the methods

The class specification given above provides the information we need in order to trace the effect of code fragments on an object of type **Turtle**. We will look now at a code fragment illustrating the effect of most of the methods. (We will look at the use of *store* and *restore* in Subsection 1.4.)

Think of the diagram as a **set** of spots. These spots are stored in a sequence, but we are not interested in the order in which they are stored, nor in whether a spot has been marked twice.

The code fragment given below introduces a new feature of the course code. This is an alternative way to add comments. A comment can be included on the same line as a code statement, with the comment preceded by `//`. (Anything following `//` up to the end of the line is part of the comment.) In the fragment below, we have used such comments to add numbers to identify the various code statements in the fragment.

```
var myTurtle in Turtle    //(1)
myTurtle.forward(5)       //(2)
myTurtle.leftTurn()       //(3)
myTurtle.forward(3)       //(4)
myTurtle.leftTurn()       //(5)
myTurtle.penOff()         //(6)
myTurtle.forward(5)       //(7)
myTurtle.penOn()          //(8)
myTurtle.forward(0)       //(9)
```

Let us look at the effect of this code fragment step-by-step. A new object of type **Turtle** is created in the default state. So after statement (1), *myTurtle* will have marker state (0,0,“right(0)”). There will be no spots in its diagram and no markers in its markerStack. The graphical representation of the turtle includes the marker’s position and direction. After statement (1), the state of *myTurtle* is as shown in Figure 1.8(a). (Note that no spots are shown in that diagram.)

Now consider the effect of statement (2). The state of *myTurtle* after that statement is executed is shown in Figure 1.8(b). The marker started at (0,0) with direction “right(0)”. Statement (2) moves it 5 places in its current direction, to end at (5,0). (The marker direction is not changed.) The default state of the pen is “on”. As the marker moves forward, all the grid points that it passes over are marked. So spots are marked at (0,0), (1,0) and so on, up to (5,0).

Now look at statement (3). The state of *myTurtle* after that statement is executed is shown in Figure 1.8(c). The marker has been turned through 90 degrees anticlockwise (to face “up(90)”).

After statement (4), the state of *myTurtle* is as shown in Figure 1.8(d). The method *forward* always moves the turtle in the current direction of the marker. The pen is still “on”, so the marker moves up 3 places, marking the points that it passes. After (4), *myTurtle* is at the position (5,3) and still facing “up(90)”.

After statement (5), the state of *myTurtle* is as shown in Figure 1.8(e). The marker has been turned another 90 degrees anticlockwise, and now faces “left(180)”.

Now consider the effect of statements (6) and (7). The state of *myTurtle* after those statements are executed is shown in Figure 1.8(f) on the facing page. The marker has been moved forward 5 places in the current direction, to end at (0,3). But because the pen was taken “off” in statement (6), no points are marked in this move.

Finally, consider the effect of statements (8) and (9). The state of *myTurtle* after those statements are executed is shown in Figure 1.8(g) on the facing page. The pen state has been returned to “on” in (8). Statement (9), *myTurtle.forward(0)*, does not move the turtle, but it does mark a spot at

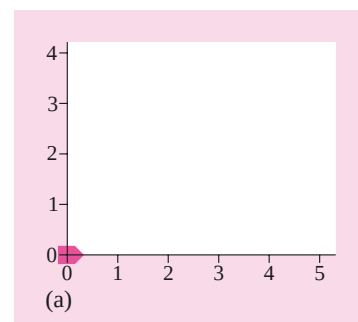


Figure 1.8 (a)

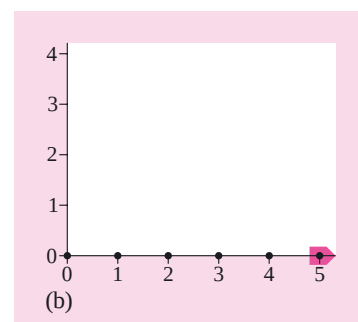


Figure 1.8 (b)

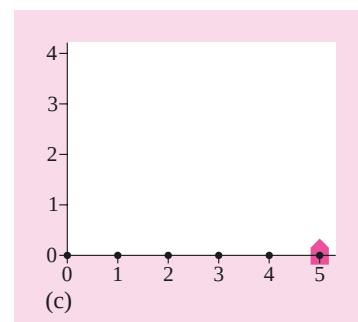


Figure 1.8 (c)

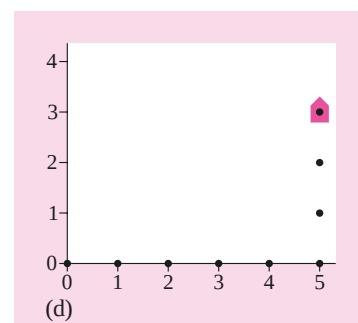


Figure 1.8 (d)

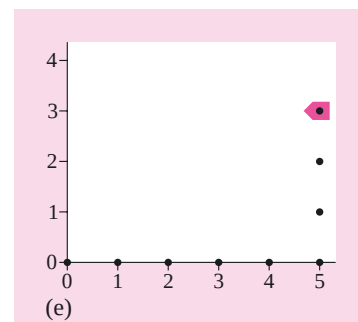


Figure 1.8 (e)

its current position. (This is the sort of detail for which it is useful to refer to the specification of the class **Turtle**.)

The figures we have given so far are all neatly represented, being based on the form shown by the WorkPad. If you are following on paper the effect of code involving a turtle, then you are likely to draw something less polished. A rough pen and paper sketch such as that in Figure 1.9(a) carries all the information of Figure 1.8(g). It shows the diagram, and the position and direction of the marker. It is usually perfectly adequate (and much quicker) to draw a sketch diagram such as that in Figure 1.9(b). We know that a turtle does not draw “continuous” lines like those used in Figure 1.9(b). So we need to interpret such a continuous line as meaning that only the grid points have been marked in the turtle diagram. But to represent on paper the effect of a statement such as *myTurtle.forward* (100), one would certainly want to draw a continuous line rather than marking 101 separate spots! If we want to emphasise that a diagram in the style of Figure 1.9(b) is an adequate response to an activity, we will ask for a “rough sketch”.

We look now at more code fragments involving a turtle. There are essentially just two techniques involved.

- The first is to follow the effect of a given code fragment in which the methods of the class are applied to a turtle, and show the resulting diagram and marker state.
- The second is the reverse problem. Given a desired diagram and final marker state, devise a code fragment that will construct the diagram and leave the marker in the given state.

1.3 From code to diagram

For code involving a turtle, it is usually the diagram that is drawn that we are interested in. Of course, to follow the effect of such code we need to keep track of all the details of the state of the turtle. We now look at some code fragments, and determine the diagram that results.

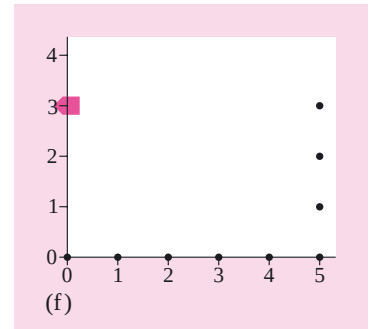


Figure 1.8 (f)

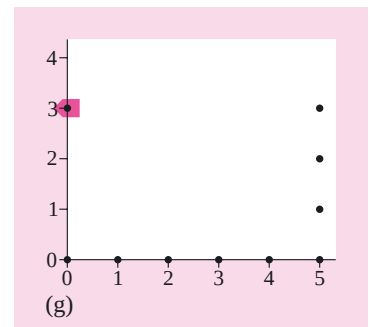


Figure 1.8 (g)

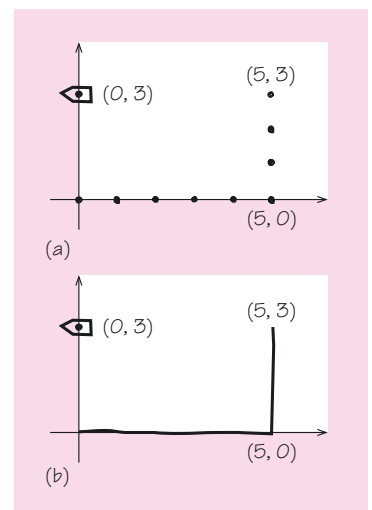


Figure 1.9 Hand drawn versions of Figure 1.8(g). (a) shows each spot that has been marked, but for many purposes, a rough sketch in the style used in (b) is adequate.

Activity 1.1

A code fragment is given below. Give the diagram that is produced by execution of this fragment. Also, give the final marker state of *myTurtle* after the fragment is executed, and show that on your figure.

```
var myTurtle in Turtle      //(1)
myTurtle.leftTurn()
myTurtle.leftTurn()
myTurtle.forward(1)
myTurtle.rightTurn()
myTurtle.penOff()
myTurtle.forward(10)
myTurtle.rightTurn()
myTurtle.penOn()
myTurtle.forward(1)
```

Discussion After statement (1), *myTurtle* is in the default state, with marker state (0,0,“right(0)”). The effect of each of the remaining code statements is indicated below.

<i>myTurtle.leftTurn()</i>	The marker state becomes (0,0,“up(90)”).
<i>myTurtle.leftTurn()</i>	The marker state becomes (0,0,“left(180)”).
<i>myTurtle.forward(1)</i>	The marker is moved 1 place in the current marker direction (“left(180)”), to (−1,0). The pen is “on” (by default), so spots are marked at (0,0) and (−1,0).
<i>myTurtle.rightTurn()</i>	The marker state becomes (−1,0,“up(90)”).
<i>myTurtle.penOff()</i>	The pen is now “off”.
<i>myTurtle.forward(10)</i>	The marker is moved 10 places up, to (−1,10). The pen is “off”, so no spots are added to the diagram.
<i>myTurtle.rightTurn()</i>	The marker state becomes(−1,10,“right(0)”).
<i>myTurtle.penOn()</i>	The pen is now “on”.
<i>myTurtle.forward(1)</i>	The marker is moved one place right. The pen is “on”, so spots are added to the diagram at (−1,10) and (0,10).

After execution of this fragment, *myTurtle* has a diagram containing four points (the state of the diagram attribute is [(0,0), (−1,0), (−1,10), (0,10)]). The marker of *myTurtle* is at (0,10) with direction “right(0)”. This state is shown in Figure 1.10.

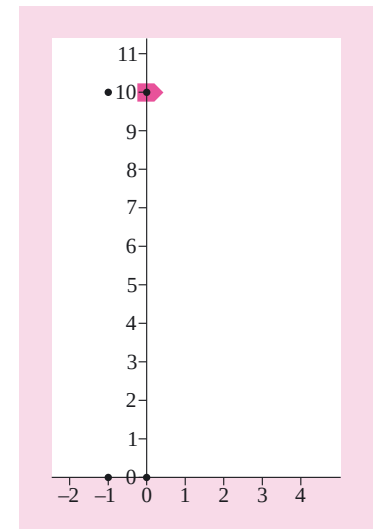


Figure 1.10

In Activity 1.1, we traced the effect of each individual code statement on the turtle. We will usually take a rather quicker approach. We also expect that when working “on paper” you will usually use sketches like that in Figure 1.9(b) to show diagrams.

Activity 1.2

A code fragment is given below. Give a rough sketch of the diagram that is produced by execution of this fragment. Also, give the final marker state of *myTurtle* after the fragment is executed, and show that on your sketch.

```
var myTurtle in Turtle  //(1)
myTurtle.leftTurn()    //(2)
myTurtle.forward(5)    //(3)
myTurtle.rightTurn()   //(4)
myTurtle.forward(10)   //(5)
myTurtle.rightTurn()   //(6)
myTurtle.forward(3)    //(7)
myTurtle.rightTurn()   //(8)
myTurtle.forward(2)    //(9)
myTurtle.rightTurn()   //(10)
```

Discussion The marker is moved as indicated below (where the numbers refer to those given in the comment in each code statement). Notice that the pen is “on” throughout execution of the code, so all points visited by the marker are added to the diagram. The diagram that is produced by this code fragment is sketched in Figure 1.11.

It starts at (0, 0) pointing “right(0)”.	(1)
It is turned to face “up(90)”.	(2)
It is moved 5 places, to (0, 5).	(3)
It is turned to face “right(0)”.	(4)
It is moved 10 places, to (10, 5).	(5)
It is turned to face “down(270)”.	(6)
It is moved 3 places, to (10, 2).	(7)
It is turned to face “left(180)”.	(8)
It is moved 2 places, to (8, 2).	(9)
It is turned to face “up(90)”.	(10)

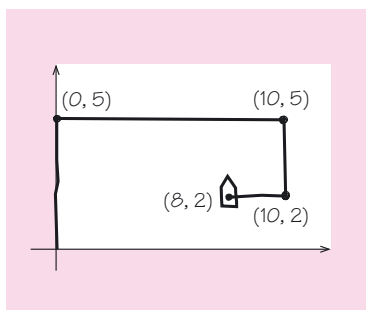


Figure 1.11

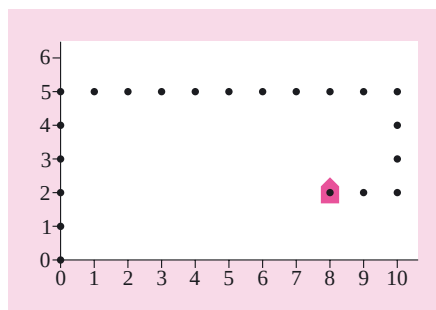


Figure 1.12

If further code followed this fragment, we would need to know the final state of the marker in full. Here the final marker state is given by the position (8, 2) and the direction “up(90)”. This final marker state is shown in Figure 1.11. (An exact representation of the final state is given in Figure 1.12.)

The next activity asks what happens if statements (2)–(10) in Activity 1.2 are repeated twice. To do this quickly, one wants to “reuse” the work that we did in Activity 1.2. The effect of the first iteration of the **for** loop in Activity 1.3 is exactly the same as that of the code in Activity 1.2, of course. To see the effect of repeating that code a second time is less easy.

Activity 1.3

A code fragment is given below. Give a rough sketch of the diagram that is produced by execution of this fragment. Also, give the final marker state of *myTurtle* after the fragment is executed, and show that on your sketch.

```
var i in Int
var myTurtle in Turtle  //(1)
/* Repeat the rest of the code in Activity 1.2 twice. */
for (i<--1 to 2)
{
myTurtle.leftTurn()      //(2)
myTurtle.forward(5)      //(3)
myTurtle.rightTurn()     //(4)
myTurtle.forward(10)     //(5)
myTurtle.rightTurn()     //(6)
myTurtle.forward(3)      //(7)
myTurtle.rightTurn()     //(8)
myTurtle.forward(2)      //(9)
myTurtle.rightTurn()     //(10)
}
```

Discussion The resulting diagram and marker state is sketched in Figure 1.13(a) (and shown neatly in Figure 1.13(b)). The first execution of the body of the **for** loop leaves the turtle in the state illustrated in Figure 1.12. Starting from there (remember the marker state is (8, 2, “up(90)”), we can trace the effect on the marker of the second execution of the body of the **for** loop as below.

It is turned to face “left(180)”.	(2)
It is moved 5 places, to (3, 2).	(3)
It is turned to face “up(90)”.	(4)
It is moved 10 places, to (3, 12).	(5)
It is turned to face “right(0)”.	(6)
It is moved 3 places, to (6, 12).	(7)
It is turned to face “down(270)”.	(8)
It is moved 2 places, to (6, 10).	(9)
It is turned to face “left(180)”.	(10)

Example 1.1 below shows the diagram obtained by repeating the code fragment in Activity 1.2 four times, which is even more complicated.

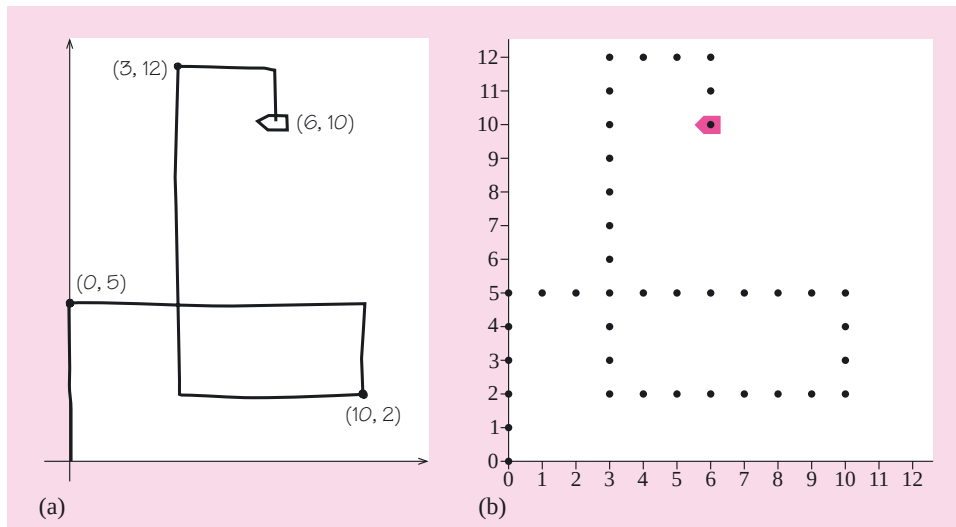


Figure 1.13

EXAMPLE 1.1

The code fragment below will leave *myTurtle* in the state shown in Figure 1.14 overleaf.

```

var i in Int
var myTurtle in Turtle
for (i<--1 to 4)
{
  myTurtle.leftTurn()
  myTurtle.forward(5)
  myTurtle.rightTurn()
  myTurtle.forward(10)
  myTurtle.rightTurn()
  myTurtle.forward(3)
  myTurtle.rightTurn()
  myTurtle.forward(2)
  myTurtle.rightTurn()
}

```

You may find this diagram difficult to sketch from the code. It is drawn as follows.

- From O to A by the first execution of the loop body.
- From A to B by the second execution of the loop body.
- From B to C by the third execution of the loop body.
- From C back to O by the fourth execution of the loop body.

You may find that you can follow how the code produces the diagram like this. First, sketch the diagram of Figure 1.14 on a piece of paper. Follow the first execution of the loop body. Rotate your paper 90 degrees clockwise. Starting at A (the point (8, 2)), trace the second execution. Rotate the paper another 90 degrees clockwise. Starting at B (the point (6, 10)), trace the fourth execution. Rotate the paper another 90 degrees clockwise. Starting at C (the point (−2, 8)), trace the third execution.

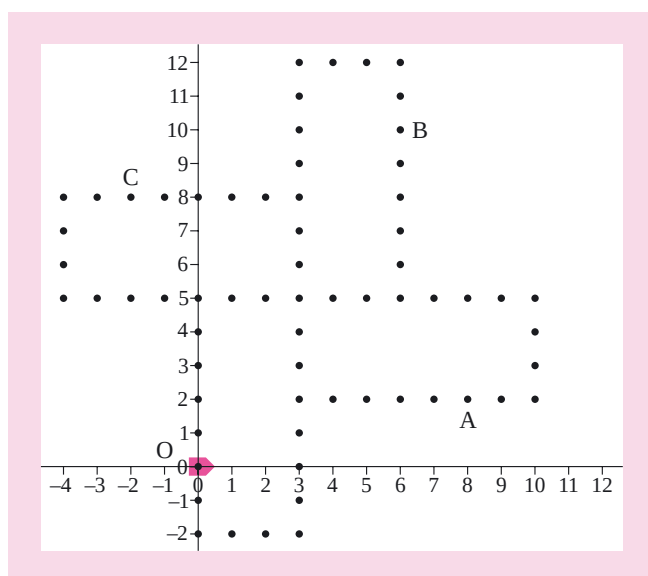


Figure 1.14

1.4 Marker state invariance

There is a particular point of interest relating to the code in Example 1.1. Notice that in Figure 1.14 the marker has finished in the default state, namely (0, 0, “right(0)”). Now before the code in the **for** loop in Example 1.1 is executed, the marker will also be in the default state (since *myTurtle* has just been declared in the **var** statement). The code in this **for** loop has not changed the state of the marker.

We say that the **for** loop in Example 1.1 is marker state invariant. More generally, we call any code fragment involving a single turtle **marker state invariant** if execution of the fragment leaves the marker state the same at the end as it was at the start. When designing code to draw complicated diagrams, it can be useful to use fragments that are marker state invariant.

If you wish to decide whether a code fragment is marker state invariant, a useful check is to count the number of turns that the marker makes during the fragment. In order for the marker direction to be the same at the end as it was at the start, the difference between the total number of left turns and the total number of right turns must be a multiple of 4. This will ensure that the marker has been turned through an integer number of complete revolutions during execution of the code fragment.

For example, in the body of the **for** loop in Example 1.1, there are 4 right turns and 1 left turn; the overall effect on the marker direction is the same as 3 right turns (or 1 left turn). This is not a multiple of 4, so the body of the **for** loop is not marker state invariant. But the **for** loop as a whole repeats these turns 4 times, resulting in a total of 16 right turns and 4 left turns. This is equivalent to 12 right turns, and 12 is a multiple of 4.

Of course, this check does not guarantee marker state invariance. For a fragment to be marker state invariant, we also require the turtle to have returned to the same marker position that it started from.

When writing code, there is a simple way in which you can ensure that a fragment (however complicated) is marker state invariant. This is to use the methods *store* and *restore*. If the first statement stores the marker state, and the last restores this state to the marker, then the code fragment will be marker state invariant.

The code has, of course, changed the state of the turtle. It has drawn quite a complicated diagram, which will be reflected in the state of the diagram attribute.

Activity 1.4

- (a) Consider the following code fragment, where *t* is an object of type **Turtle**, and is in the default state when the fragment is encountered.

```
t.store()      //(1)
t.forward(5)   //(2)
t.restore()    //(3)
```

- (i) Give the state of the marker after this fragment has been executed.
(ii) Is this fragment marker state invariant?
(iii) Is the state of the turtle the same at the start and end of the code?
- (b) Suppose that the statement below is added to the end of the fragment above. Is the resulting code fragment marker state invariant?

```
t.leftTurn()   //(4)
```

Discussion

- (a) (i) At the start the marker is in the default state (0,0,“right(0)”). This is the marker state stored in the markerStack by statement (1). The marker state is changed by statement (2), but statement (3) returns the marker state to the last state stored in the markerStack (in this example, the only state stored), which is the default state.
(ii) The marker state is the same after the code fragment is executed as it was at the start. So this fragment is marker state invariant.
(iii) The state of the turtle has changed. Statement (2) adds some spots to the diagram, so this attribute, which is part of the turtle’s state, has changed.
- (b) With statement (4) included, the marker will be pointing “up(90)” after the fragment has been executed. The revised fragment is therefore not marker state invariant.

As illustrated in Activity 1.4, if you want to write a code fragment that is marker state invariant, a safe approach is to have *store* as the first statement of the fragment and *restore* as the last statement.

1.5 From diagram to code

We now look at problems of the form: given a diagram, supply a code fragment that will produce it. In addressing such a problem, it is sensible first to outline your strategy for using a turtle to draw the diagram, then to write your code.

Activity 1.5

Declare an object *myTurtle* and give a code fragment that will produce the diagram shown in Figure 1.15.

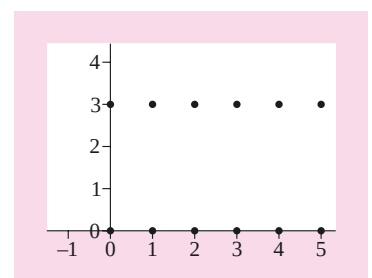


Figure 1.15

Discussion There are many ways to produce the required diagram. We will use the following strategy.

1. Plot the lower line.
2. Turn the pen off and go to the point (5,3) at the end of the top line.
3. Turn the pen “on” and plot the top line.

```
var myTurtle in Turtle // in default state
myTurtle.forward(5)    // plot the line to (5,0)
myTurtle.penOff()      // turn the pen off
myTurtle.leftTurn()    // turn to point "up(0)"
myTurtle.forward(3)    // move to the point (5,3)
myTurtle.penOn()       // turn the pen on again
myTurtle.leftTurn()    // turn to point "left(180)"
myTurtle.forward(5)    // plot the line to (0,3).
```

Activity 1.6

Give a code fragment that will produce the diagram shown in Figure 1.16 when applied to a turtle t in the default state. Give a fragment that is marker state invariant.

Discussion The turtle will start in the default state, which is at (0,0) and pointing “right(0)”. Use the following strategy to draw the diagram.

1. Draw the bottom line, from (0,0) to (5,0).
2. Turn left, and draw the line from (5,0) to (5,5).
3. Turn left, and draw the line from (5,5) to (0,5).
4. Turn left, and draw the line from (5,0) to (0,0).

This strategy leads to the following code.

```
t.forward(5)          // plot line to (5,0)
t.leftTurn()          // direction is now "up(90)"
t.forward(5)          // plot line to (5,5)
t.leftTurn()          // direction is now "left(180)"
t.forward(5)          // plot line to (0,5)
t.leftTurn()          // direction is now "down(270)"
t.forward(5)          // plot line to (0,0)
```

This code leaves the marker at (0,0). However it is pointing “down(270)”, while its direction at the start of the fragment was “right(0)”. So this fragment is not marker state invariant. We could use *store* and *restore* to ensure marker state invariance, but we can use a neater approach in this example. Add a final statement

```
t.leftTurn()
```

This will ensure that the final direction of the marker is “right(0)”, and so the fragment is marker state invariant.

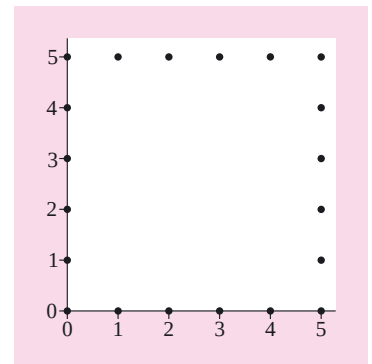


Figure 1.16

With this final *leftTurn* statement added, we can see the code fragment in the above discussion to be a repetition of two statements four times. So the code below will draw the diagram in Figure 1.16.

```
var t in Turtle
var i in Int
for (i<--1 to 4)
{
    t.forward(5)
    t.leftTurn()
}
```

The **for** loop here is marker state invariant.

As illustrated in Figure 1.14, a turtle can produce quite complex diagrams. We explore systematic ways of drawing complicated diagrams in Section 3.

Objectives for Section 1

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: marker, pen, diagram, markerStack, marker state and marker state invariant.
- ☐ Give the change in state produced when a method of the class **Turtle** is applied to an object of that class (in a known state).
- ☐ Given a code fragment involving an object of type **Turtle**, produce the graphic representation of the final state of the turtle, showing both its diagram and the marker state.
- ☐ Given a diagram, produce a code fragment that will draw that diagram.
- ☐ Decide whether a given code fragment is marker state invariant.
- ☐ Use the methods *store* and *restore* to ensure that a code fragment is marker state invariant.

Exercises on Section 1

Exercise 1.1

The following code has been executed to draw the diagram in Figure 1.16.

```
var t in Turtle
var i in Int
for (i<--1 to 4)
{
    t.forward(5)
    t.leftTurn()
}
```

The following additional code statements are then executed. Sketch the resulting diagram, and write down the final marker state of *t*.

```
t.store()           //(1)
t.forward(10)       //(2)
t.rightTurn()       //(3)
t.forward(5)        //(4)
t.restore()          //(5)
```


Exercise 1.2

- (a) Write a fragment of code that, when applied to a turtle *t* in the default state, will draw the diagram shown in Figure 1.17, leaving the marker state as shown in that figure.
- (b) Call the code you wrote for part (a) *Fragment1*. Suppose that *Fragment1* is used as the body of a **for** loop, as indicated below. What will be the state of the turtle *t* after the code below is executed?

```

var t in Turtle
var i in Int
for (i  $\leftarrow$  1 to 3)
{
    Fragment1
}

```

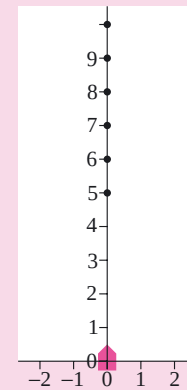


Figure 1.17

2 Implementing the class *Turtle*

In Section 1 we discussed the specification of the class **Turtle**. We described the form of data that needs to be stored to represent the state of a turtle, and looked at the methods of **Turtle**, and how these may be used to produce diagrams. In designing code using a turtle to draw some particular diagram, all we need is given in the specification of the class and its methods. The specification provides all the information needed in order to use any class. For instance, to design code to get a turtle to draw some given diagram we do not need to know how the method *forward* is implemented for the class **Turtle**, but only the effect *forward* has on a turtle: how it moves the marker and what it may add to the diagram.

In Unit 4 we described how the classes introduced there can be implemented, using only the primitive data types. We gave the implementations in a form suitable for entry into the WorkPad. However, to use these classes in the WorkPad we do not need to enter their implementations. The WorkPad has a **library** of classes that have already been implemented. The library includes **Spot**, **QuickSpot** and **ColourSpot**. The class **Turtle** is also in the WorkPad library. So when working in the WorkPad, **Turtle** is already available as an implemented class.

However, it is instructive to consider some aspects of how **Turtle** is implemented. What we have available from Block 1 are the primitive types described in Unit 3, together with the spot classes discussed in Unit 4. Now details of the code required to implement some of the methods of **Turtle** is complicated, but involves nothing that is new in principle. So we shall not give implementations of all of the methods of **Turtle** here. In this section, we shall look in particular at the attributes needed to represent the data set, and how these have been implemented. In doing this, we will take the opportunity to consider some more general issues that may arise when we progress from a class specification to an implementation.

2.1 The attributes

In Section 1, we noted that the state of a turtle is determined by four features: the marker; the pen; the diagram and the markerStack.

In choosing how to implement the data set of **Turtle**, it is natural to have an attribute that reflects each of these four features. (Indeed, it is difficult to describe the data set of any class in a fully abstract way. The specification will usually suggest the attributes that are used in the implementation.) In the implementation, we will name the attributes as follows:

attMarker (for the marker);
pen (for the pen);
diagramStack (for the diagram);
markerStack (to store marker states).

Each of these attributes needs to be implemented using an available type. The pen has only two possible states, “on” and “off”, so we can represent the *pen* attribute using **Bool**. We will take **true** to represent “on” and **false** to represent “off”.

The marker has a position and a direction on a grid of points. None of the available primitive types or classes combines both these features. The class **Spot** has a position, and it has four methods that move it one grid position at a time in one of the four required directions. Also, a turtle has to be able to record each of the positions on the grid that it passes over (by adding them to the diagram). So a spot with its “one at a time” method of changing position on the grid seems ideal for the marker’s position and movement criteria. In the next subsection, we shall specify a new class, called **Marker**, to implement the attribute *attMarker* of a turtle. A marker is a spot, with an additional feature (a direction). We shall specify **Marker** as an extension of the class **Spot**. (You met the **extends** relationship between classes in Unit 4.)

The fourth attribute of a turtle, *markerStack*, enables us to store marker states. This is a sequence, which will need to be able to change size: to grow when a new marker is stored, and to shrink when *restore* is called and a marker is deleted from *markerStack*. The methods will only need to access one end of the sequence, so a stack of markers will provide a convenient implementation of *markerStack*.

The remaining attribute of a turtle is the diagram. Each point on the diagram can be implemented as a spot. The entire diagram is a collection of spots. As an abstraction, we might specify the diagram as a set of spots. However, we do not yet have available a class to represent a set. The only type we currently have available to represent a collection whose size varies is **Stack of X**. So the attribute *diagramStack* used to store the diagram is implemented as a stack of spots.

2.2 The class Marker

The marker feature of a turtle has a position and a direction on a grid of points. We will now specify a class **Marker** that we can use to implement the attribute *attMarker* of **Turtle**. This class **Marker** is designed for the specific purpose of representing one attribute of the class **Turtle**.

In the specification below, notice that **Marker** extends **Spot**. This means that a marker object inherits the methods and attributes of the class **Spot**. A marker has an additional attribute: a direction. The methods of **Spot** provide what is required to move a marker object around the grid. Each move is in one of the four possible marker directions. A spot moves one position at a time, which will be helpful when recording the positions visited by the marker.

Marker	Class specification
Marker extends Spot	
<i>States</i> A marker has a position on a two-dimensional grid (as in Spot) together with a direction.	
<i>State notation</i> (x, y, n) where x is the x -coordinate of the marker, y is the y -coordinate of the marker and n is the direction of the marker.	
<i>Default</i> The marker is in the state $(0, 0, \text{"right(0)"})$.	
<i>Methods</i>	
<i>nextDirection()</i> state change	
plus from Spot	
<i>getXPos, getYPos, upOne, downOne, leftOne, rightOne.</i>	

The class **Marker** has one new method, whose specification is given below.

method *nextDirection()* **state change**
pre **true.**

post The marker's direction attribute is changed so as to rotate the marker 90 degrees anticlockwise. (So "right(0)" is changed to "up(90)", "up(90)" to "left(180)", "left(180)" to "down(270)" and "down(270)" to "right(0)".)

To implement the class **Marker**, we need to give implementations of any additional methods and attributes that it has. As well as the two attributes *xPos* and *yPos* inherited from **Spot**, which give the position, a marker has an additional feature, its direction. To implement this, a marker object has an additional attribute, which we will call *direction*. This has type **Int**, with the only possible values being 0, 1, 2 or 3. We represent the direction of a marker using an integer, as below (see Figure 2.1).

"right(0)" is represented by 0.

"up(90)" is represented by 1.

"left(180)" is represented by 2.

"down(270)" is represented by 3.

We can give an implementation of the method *nextDirection* as below. Recall from Unit 4 that in implementing a method we use *this* to refer to the object to which the method is applied (the receiver). In a method implementation, we can access the attribute *direction* of the receiver as *this.direction*.

method *nextDirection()*
 $\{ \text{this.direction} \leftarrow \text{MOD}(\text{this.direction} + 1, 4) \}$

Compare this implementation to the code for the method *nextColour* in the class **ColourSpot**. As there, we need to cycle through the numbers 0, 1, 2 and 3, by adding 1 to the number representing the previous direction. If this becomes 4, then we must return to the value 0 again. Here, we achieve this by using the *MOD* function for integers.

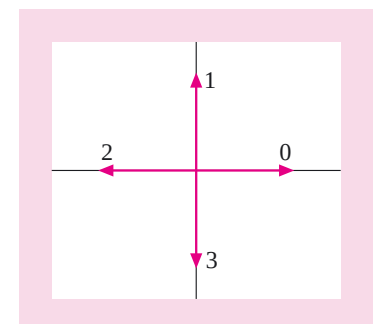


Figure 2.1 The representations of the four possible marker directions.

See Unit 4, p. 37.

Notice how the class **Marker** has developed from the specification of the class **Turtle**, and the need to implement its marker feature (the attribute *attMarker*). We needed to make available a class to implement the attribute *attMarker*, using available types. For the marker, we required something that behaved like a spot object (in that it has a position on a grid and can be moved one position at a time). We also needed an additional attribute giving the marker direction, coupled with a single additional method to change direction. We achieved this by specifying a class **Marker** that **extends** **Spot**. Use of extension here is particularly suitable because we want a marker to have access to all the methods of the class **Spot**.

Because of the **extends** relation between the classes, we may say that a **Marker** object “is a spot”. (A marker has an additional attribute and method, of course.)

2.3 Composition of classes

In the specification of **Marker**, we chose to say that **Marker** **extends** **Spot**. In using the **extends** relation for our new class (**Marker**) we make decisions about both the specification and implementation of the new class. It will have access to all the attributes and the methods of the old class (**Spot**). So in specifying that **Marker** **extends** **Spot**, we make a decision that a marker object will have the attributes of a spot as part of its state and also that it will respond to the methods of **Spot**.

The main benefit of using the **extends** relation is this.

- Code is reused in the implementation of the new class. The class **Marker** uses existing code from **Spot**.

This both saves programming effort and improves reliability, since the code from **Spot** will already have been tested. This is particularly important. However, as we shall see shortly, extension is not the only way of achieving reuse of code.

Another key feature of extension concerns user code.

- Any code that uses a marker can use the methods of **Spot**, which may enable one to simplify the code.

Looked at another way, this same feature might be considered a drawback!

- The new class inherits **all** the methods of the old class. This means a proliferation of methods (the old added to the new) available to a user. This may be beneficial or it might simply add to the complexity faced by a user of the class.

Indeed, there are situations where a new class has the same form of data as an existing class, but where one does not want some of the methods of the existing class to be available to the new class. We may want to **hide** some of the existing methods from the new class. In such a situation, use of extension is not appropriate.

One useful guide as to when to use extension is this.

- Use extension where this reflects a relationship of specialisation between classes.

That is, the new class should be like the old class but more specialised, in that it has additional data or functionality. For example, a marker object is a special type of spot, with added structure and functionality (as was a coloured spot in Unit 4).

There is another way to incorporate the key benefit of extension (reuse of code), that does not add to the proliferation of methods. This is called **composition** of classes. We use composition to implement the class **Turtle**.

In Unit 4, you saw how **Spot**, for example, is implemented using primitive types to implement the attributes. We might say that a spot is “composed” of two integers *xPos* and *yPos*. The type functions of **Int** are **not** available to an object of type **Spot**. (We can use them to implement the methods of **Spot**, but in user code involving spots only the methods in the specification of **Spot** are available.)

The first task in implementing any class is to decide how to describe the attributes in terms of what we already have. To start with, the primitive data types, **Int**, **Char**, **Bool**, together with arrays and stacks of these types were available. But we now also have available certain classes. We have described how classes such as **Spot** and **Marker** can be built up from the primitive types. So an attribute can be chosen to be implemented as one of these available classes.

Recall that the four attributes of the class **Turtle** are *attMarker*, *pen*, *diagramStack* and *markerStack*. We have indicated a type that can be used to implement each of these attributes.

In Subsection 2.1.

Now we might have chosen to say that **Turtle extends Marker**. But if we do this, then all the methods of **Marker** will be available to **Turtle**. In particular **Turtle** would have methods such as *upOne*, that **Marker** inherits from **Spot**. We do not want this. The method *upOne* would add no useful functionality to the class **Turtle**, but would lead to an unnecessary proliferation of methods available to a turtle.

Rather than having **Turtle extends Marker**, we can implement the attribute *attMarker* using the class **Marker**. This is an example of composition of classes. We say that **Turtle** is **composed** with a **Marker** object. This is the first example we have given where an attribute is drawn from an existing class (rather than a primitive type).

As suggested earlier, we implement the attributes of **Turtle** as follows.

attMarker has type **Marker**.

pen has type **Bool**.

diagramStack has type **Stack of Spot**.

markerStack has type **Stack of Marker**.

Using composition does not lose the key benefit of extension that we noted above. Indeed, for composition we have the following.

- Code is reused. The code to implement the methods of **Turtle** will make use of the code already written for **Marker** and **Spot** through the attribute *attMarker*.
- Instead of a turtle being a special type of marker, a turtle “has” a marker.
- User code cannot treat a turtle as a marker, but the code that implements the methods of a turtle can treat *attMarker* both as a marker and a spot.

There is still a significant relationship between the classes **Turtle** and **Marker** (and hence **Spot**). The major difference between extension and composition is that the latter does not supply a turtle object with all the methods of **Marker** (and so of **Spot**). The class **Turtle** has only the methods of the class described in the specification in Section 1.

Activity 2.1

To which of these code statements will an object *myTurtle* of type **Turtle** respond?

- (a) *myTurtle.upOne()*
- (b) *myTurtle.forward(1)*
- (c) *myTurtle.nextDirection()*
- (d) *myTurtle.leftTurn()*
- (e) *myTurtle.clone()*

Discussion Since *myTurtle* is of type **Turtle**, it will only respond to the methods specified for the class **Turtle**, which are those in the statements in (b), (d) and (e). (Remember that *clone* is available to an object of any M263 class.)

To recap, composition uses an old class to implement an attribute of a new class. It is a very different approach to that used by extension.

- In composition, any object of the new class “has” an object of the old class. The new class only has the methods directly specified for it.
- In extension, any object of the new (child) class “is” an object of the old (parent) class. The methods of the parent class are added to those of the child class.

Composition is an object-oriented design concept for building objects from other objects. When should you use it? A very rough rule of thumb is this: the presence of “has” or “have” in an informal description of the new class may indicate a need for composition. Use composition when:

you want to limit the new class to its own methods and not to provide it with those of an old class as well;
 you need the functionality of the old class to implement the new class methods.

Extension builds new classes from old ones in layers. Each new layer has all the previous attributes and methods, with some new features added.

It is **not** an aim of this course that you learn to design your own classes.

2.4 Using composition — an example

The class we shall discuss in this subsection is included here as an example of how we can implement in the WorkPad a new class that uses composition. Here is a rough description of this new class, which we call **HLine**. An object of this class will have a position on a grid, like a spot, and will be able to draw a horizontal line of points, starting at its current position.

The state of an object of type **HLine** will be determined by two features: a marker position and a diagram. The class **HLine** has the following methods.

moveTo(*x*, *y*), which moves the marker to the grid position with coordinates (*x*, *y*).

draw(*n*), which adds a horizontal line of spots, of length *n*, to the diagram.

HLine is not in the class library in the WorkPad.

Activity 2.2

Suggest specifications of each of the methods of the class **HLine**.

Discussion Here are suitable specifications for the methods. For *draw*, the informal description given above does not give the effect on the marker feature of the receiver. We have chosen to specify that *draw* leaves the marker where it was before *draw* was called. (You may well have suggested that the marker is moved to the end of the new line. That is a different, but equally acceptable, interpretation of the informal description given above.)

method *moveTo*(*x* in Int, *y* in Int) **state change**

pre true.

post The marker is moved to the position (x, y) . (The diagram is not changed.)

method *draw*(*n* in Int) **state change**

pre $n \geq 0$.

post A line of length *n* is added to the diagram (consisting of $n + 1$ spots), starting at the current position of the marker, and going horizontally right. (The marker position is not changed.)

Here is a specification of **HLine** (its methods are specified in the above discussion).

HLine	Class specification
<i>States</i> The state has two features: a marker position on a two-dimensional grid, and a diagram (that can be thought of as a set of spots).	
<i>State notation</i> The marker position is given as a pair (x, y) . The diagram will usually be shown graphically.	
<i>Default</i> Marker at the origin and diagram empty.	
<i>Methods</i>	
<i>moveTo</i> (<i>x</i> in Int, <i>y</i> in Int) state change	
<i>draw</i> (<i>n</i> in Int) state change	

Activity 2.3

For the following code, describe the state of *m* after each statement.

```
var m in HLine    //(1)
m.moveTo(-1, 2)   //(2)
m.draw(3)         //(3)
```

Discussion After (1) the object *m* is in the default state, at $(0, 0)$ with empty diagram.

After (2) *m* is at $(-1, 2)$ (diagram still empty).

After (3) *m* is still at $(-1, 2)$. The diagram now consists of the points $(-1, 2)$, $(0, 2)$, $(1, 2)$ and $(2, 2)$ (see Figure 2.2).

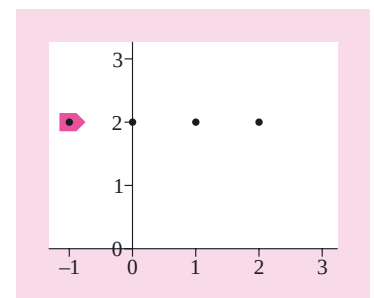


Figure 2.2

Activity 2.4

Suggest a way of implementing the data set of **HLine** that makes use of an existing class. What attributes would you use, and what type would you give them? Would you use extension or composition in specifying **HLine**?

Discussion An object of type **HLine** is to have a grid position and a diagram. A turtle has these features (and a lot more). But we need to use **Turtle** in composition rather than to use extension. If we use a single attribute that is a turtle, then we can represent the data set of **HLine**. At the same time, we can hide the additional structure and functionality that is available to an object of type **Turtle**.

We will implement **HLine** using a single attribute *attTurtle* in **Turtle**. The diagram of **HLine** will be the diagram attribute (*diagramStack*) of **Turtle**, while the marker position of **HLine** will be the marker position of the marker attribute (*attMarker*) of **Turtle**.

In Unit 4, we showed how to implement the class **Spot** in a form suitable for the WorkPad. There, we gave statements assigning default values to each attribute. For **HLine**, we have a single attribute *attTurtle*, and its default state can be inherited from the class **Turtle**. That will give a default state for **HLine** with marker position (0,0) and diagram empty, as in the specification. For the WorkPad, this is achieved by including this statement.

```
this.attTurtle in Turtle
```

The method *draw* of **HLine** can be implemented using *forward* (but we must be careful to return the marker to its original position, which we can do using *store* and *restore*).

The method *moveTo* can be implemented using methods of **Turtle**. However, this is a little awkward, since **Turtle** has no methods that allow us to directly set the marker position. We shall use a different approach, which is available when implementing a class by composition.

In code implementing a method of **HLine**, one can access the value of the attribute *attTurtle* as *this.attTurtle*. This attribute is an object of type **Turtle**, to which we can apply methods of that class. However, we can write much simpler code for *moveTo* if we are able to access the attributes of the turtle itself (that is, the attributes of the attribute of the receiver object). The course code allows us to do this in the context of a class implemented by composition. For example, the marker attribute of the attribute *attTurtle* of the receiver *this* is accessed as *this.attTurtle.attMarker*. (Recall that the class **Turtle** has an attribute *attMarker* that is of type **Marker**.)

Now **Marker** extends **Spot**, and the attributes of an object of type **Marker** are *xPos*, *yPos* and *direction*. It is the values of *xPos* and *yPos* that we want to set in implementing *moveTo*.

Not all programming languages would permit the approach we use here. Some would be more restrictive in the way access to attributes is allowed. (We have included this more relaxed approach to illustrate what is permitted in some programming languages.)

Where an attribute is of class type, we will usually use the prefix “att” in naming it. This will avoid name clashes between the attribute and its type.

Remember that in the context of a method implementation, the receiver object is always identified as *this* in the course code.

In the course code, access to attributes is *only* permitted when implementing a new class. It is not permitted either in fragments of “user code”, or when implementing a function.

EXAMPLE 2.1

To implement the method *moveTo*(*x*,*y*) of **HLine** we need to set the values of the attributes *xPos* and *yPos* of the attribute *attMarker* of the attribute *attTurtle* of the receiver object of type **HLine**. Each needs to be set to the value of the appropriate parameter. The following code statements do that.

```
this.attTurtle.attMarker.xPos<--x
this.attTurtle.attMarker.yPos<--y
```

When we implement the class **HLine** in the WorkPad, we provide a separate identifier for the code implementing each of the methods. There will be statements

```
this.moveTo calls moveToHLine
this.draw calls drawHLine
```

and the code for the implementation of each method will come later. For example, the implementation of *moveTo* will be given as follows.

```
method moveToHLine(x,y)
{
  this.attTurtle.attMarker.xPos<--x
  this.attTurtle.attMarker.yPos<--y
}
```

Activity 2.5

Give code that could be used for the method implementation *drawHLine* of the class **HLine**. (Note that the marker direction of the attribute *attTurtle* of an object of type **HLine** will always have the default value “right(0)”, since no method of **HLine** will change that.)

Discussion To implement *draw*, we can use *forward* to add a line of spots to the diagram, and *store* and *restore* to ensure that the marker position is not changed by the method.

```
method drawHLine(n)
{
  this.attTurtle.store()
  this.attTurtle.forward(n)
  this.attTurtle.restore()
}
```

Note how use of composition enables us to access the methods of **Turtle** within the implementation of **HLine**, and how the implementation of *draw* reuses code already written in implementing *forward* within **Turtle**. Note also that user code involving **HLine** has no access to the methods of **Turtle**.

2.5 Implementing methods of *Turtle*

The class **HLine** exemplifies how methods can be implemented in a class that uses composition. The same approach can be used in implementing the class **Turtle**. We look here at an example of this. We will not give implementations of most of the methods of **Turtle**. We deal here only with the methods for changing the marker direction, *leftTurn* and *rightTurn*.

A turtle object has an attribute *attMarker*. That attribute will respond to the methods of **Marker** (which include those of **Spot**). So when implementing the methods of **Turtle**, one can use statements such as these.

```
this.attMarker.rightOne()
this.attMarker.nextDirection()
```

EXAMPLE 2.2

To implement the method *leftTurn*, we use the attribute *attMarker*. The method turns the marker 90 degrees anticlockwise. To do that, we need only to use the **Marker** method *nextDirection* once to change the state of *attMarker*.

```
this.attMarker.nextDirection()
```

Activity 2.6

Write code to implement the method *rightTurn* of **Turtle**.

Discussion The only method of the class **Marker** that changes the direction is *nextDirection*. That will turn *attMarker* through 90 degrees anticlockwise. So we will need to apply that method three times. That will provide a turn through 270 degrees anticlockwise, which is equivalent to the turn 90 degrees clockwise that we want.

```
this.attMarker.nextDirection()
this.attMarker.nextDirection()
this.attMarker.nextDirection()
```

We will not give detailed code for other methods. Here is an indication of how the more complicated methods can be approached.

For methods that change the state of the marker, we use the attribute *attMarker*. For example, if the marker is facing “right(0)” then to implement *forward* we can use the method *rightOne* of **Spot**. (For the other three directions, we need to use *upOne*, etc, as appropriate.) If the pen is “on”, then, as *forward* is executed, we can push the current position of *attMarker* onto the *diagramStack* at each step, so as to update the diagram.

When implementing the method *store*, we will need to push the current value of the attribute *attMarker* onto the *markerStack*. For *restore*, we need to peek at *markerStack*, and adjust the marker position accordingly, and then pop *markerStack*. Notice in this case how we need to use the type functions of **Stack** on *markerStack*, corresponding to the type of that attribute.

Aside

The course code only permits direct access to the attributes of an object within the implementation of the class and its methods. In code using objects of a given class, access to the attributes can only be achieved by using *get* and *set* methods, if these have been included in the class specification.

In programming languages, the freedom of access to the attributes varies between languages, and also depends on whether they have been designated as **private** or **public**. It is usual that all attributes are designated **private**, meaning they cannot be accessed outside of the class. If we wish to see them and change them, then *get* and *set* methods should be included in the class specification. All methods are usually designated **public** meaning that they can be used both inside and outside of the class.

Suppose that *myTurtle* is an object of type **Turtle**. A statement such as (1) below is not valid in the course code, because user code is not permitted access to attributes such as *attMarker*.

```
myTurtle.attMarker.rightOne()    //(1)
```

However, some programming languages would allow such access. Where permitted, the effect of this statement will be to change the state of the marker feature of *myTurtle* by moving it one position to the right.

End of Aside

Objectives for Section 2

After studying this section you should be able to do the following.

- ☐ Understand the use of class composition in implementing a new class with an attribute drawn from another (existing) class.
- ☐ Understand the difference between composition and extension.
- ☐ Give an implementation of a method in the context of the implementation of a class that uses composition.
- ☐ Describe the complete implementation of a new class using composition (in simple cases).

Exercises on Section 2

Exercise 2.1

- (a) Give a code statement that could be used to implement the method *penOn* when implementing **Turtle**.
- (b) Write all the code statements that are needed in an implementation of **Turtle** to implement the method *penOff*, including statements to identify that code, and to link the method to its implementation by a **calls** statement.

Exercise 2.2

Suppose that you want a class that has all the features of a turtle, and also is able to erase a spot from the diagram. Would you expect to use extension or composition in implementing such a class?

Exercise 2.3

Put together the code for the implementation of the class **HLine** in a form suitable for entry into the WorkPad.

A method that returns the value of an attribute is known as a **get** method. A method that changes the state of an object by providing values for one or more of the attributes is known as a **set** method.

3 Extending the class *Turtle*

In Example 1.1, you saw code that uses a turtle to draw quite a complicated diagram, sketched in Figure 3.1. Notice how this is made up of four rectangles. If we had code that would draw any specified rectangle, then we could use that to design a simple strategy to produce such a diagram. Code to produce a rectangle can be specified as a function.

We start this section by looking at such a function. We then go on to examine how we can increase the functionality of our class **Turtle** by extending it, first to add a method that draws rectangles, then to add a method that draws diagonal lines.

3.1 Rectangular patterns

In Activity 1.6 we saw how to use a turtle to produce a diagram that was a particular square. Now the methods of the class **Turtle** enable a turtle to add horizontal and vertical lines of spots to a diagram. So it is a relatively simple matter to obtain a square or a rectangle using a turtle. However, if we specify and implement a function that draws rectangles, then we can reuse the work that goes into producing the code for one particular rectangle so as to be able to draw a rectangle of any size and at any place on the grid.

We first specify the function.

function *RECTANGLE*(*t* in **Turtle**, *side1* in **Int**, *side2* in **Int**) **return** in **Turtle**

pre *side1* ≥ 1 and *side2* ≥ 1 and *t* has pen state “on”.

post The returned value is a turtle in the same state as *t* except that a rectangle has been added to the diagram. The first side of the rectangle is drawn by starting at the marker position of *t*, and drawing a line of length *side1* (i.e. of *side1* + 1 spots) in the marker direction of *t*. The next side is drawn after first turning 90 degrees anticlockwise, and has length *side2*. (Then draw two more sides.)

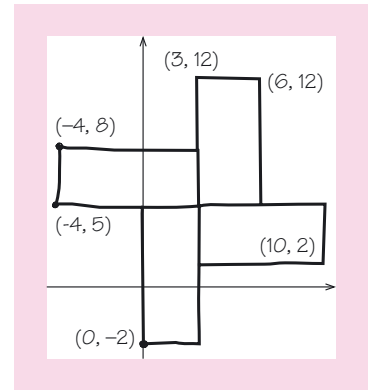


Figure 3.1

Activity 3.1

What is the state of the turtle *t* after execution of the code fragment below?

```
var t in Turtle           //(1)
t.leftTurn()              //(2)
t.leftTurn()              //(3)
t<--RECTANGLE(t,2,5)      //(4)
```

Use of *clone* is not necessary when assigning the value returned by a function to an object, as noted in Unit 4.

Discussion After (1), the marker state is (0,0,"right(0)"). Statements (2) and (3) turn the marker twice, to the state (0,0,"left(180)"). Then (4) draws a rectangle whose first side is of length 2, and runs from (0,0) to (−2,0). Its second side is of length 5, and runs from (−2,0) to (−2,−5). The resulting diagram is shown in Figure 3.2.

Activity 3.2

Give an implementation of the function *RECTANGLE* specified above.

Discussion We need to repeat the following four steps twice.

Draw a line of length *side1*.

Turn 90 degrees anticlockwise.

Draw a line of length *side2*.

Turn 90 degrees anticlockwise.

This will take the marker back to both its original position and direction.

```
function RECTANGLE(t,side1,side2)
{
  var i in Int
  var temp in Turtle    //(1)
  temp<--t.clone()      //(2)
  for (i<--1 to 2)
  {
    temp.forward(side1)
    temp.leftTurn()
    temp.forward(side2)
    temp.leftTurn()
  }
  return temp
}
```

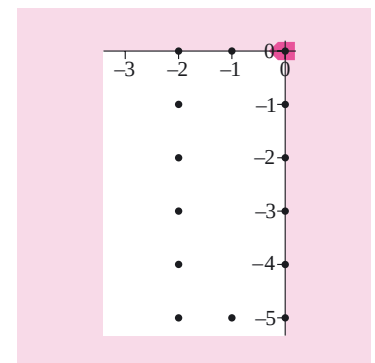


Figure 3.2

The need to include statements like (1) and (2) when implementing a function was discussed in Unit 4, Section 2.

An ability to draw rectangles would make it much easier to design code to produce a diagram such as that in Figure 3.1. We can make this functionality available in a systematic way by incorporating it as a method into a new class. An ability to draw rectangles would be a useful *addition* to the class **Turtle**. We would want to retain all the existing features of the **Turtle** class, but also be able to draw rectangles easily. This is a clear case for extending the class **Turtle**.

So we will now specify a new class (**TurtlePlus**) that **extends Turtle** (and so has all its methods), and adds a new method for plotting rectangles.

TurtlePlus	Class specification
TurtlePlus extends Turtle <i>State, State notation and Default are as for Turtle.</i> <i>Methods</i> <i>rectangle(side1 in Int, side2 in Int) state change</i> plus all the methods of the class Turtle.	

A specification of *rectangle* is given below. Remember that we are now concerned with a method of the class, not a function. There is no parameter that is a turtle; the method will always be addressed to a receiver that is an object of type **TurtlePlus**.

method *rectangle(side1 in Int, side2 in Int) state change*

pre *side1* ≥ 1 and *side2* ≥ 1 and the receiver has pen state “on”.

post A rectangle is added to the diagram (and the state of the receiver is otherwise unchanged). The first side of the rectangle is drawn by starting at the current marker position of the receiver, and drawing a line of length *side1* in the current marker direction. The next side is drawn after first turning 90 degrees anticlockwise, and has length *side2*. (Then draw two more sides to complete the rectangle.)

Activity 3.3

Give the code that would be needed to implement this new class **TurtlePlus** in the WorkPad.

Discussion We first give the necessary code, then discuss it.

```

Class TurtlePlus                                     //(1)
{
  this.rectangle calls rectangleTurtlePlus          //(2)
}

TurtlePlus extends Turtle                           //(3)

method rectangleTurtlePlus(side1,side2)              //(4)
{
  var i in Int
  for (i<--1 to 2)
  {
    this.forward(side1)
    this.leftTurn()
    this.forward(side2)
    this.leftTurn()
  }
}
// (5)

```


At (1) the new class is given its name (identifier).

At (2) the additional method of the new class is declared. On the left-hand side is the method name, *rectangle*. When the method is called, the receiver object invokes the method implementation with the identifier *rectangleTurtlePlus*.

At (3) the new class **TurtlePlus** is declared to **extend Turtle**, and so to inherit all the attributes and methods of **Turtle**.

At (4) we see that the method *rectangle* has two parameters. (These two integer parameters, *side1* and *side2*, give the lengths of the sides of the rectangle.) This is a state changing method, so we do not have to clone a temporary object at the start of the code, and there is no value to be returned at (5). This is different to the function *RECTANGLE*, which did return a turtle. The effect of the code in *rectangleTurtlePlus* is to add a rectangle to the diagram of the receiver object (*this*).

An object of type **TurtlePlus** now has the following “drawing” methods. It can draw:

a single dot (*forward(0)*);
 a line (*forward(n)*);
 a rectangle (*rectangle(side1,side2)*).

Activity 3.4

Give a rough plot of the diagram of the **TurtlePlus** object *t* after the code fragment below has been executed.

```
var t in TurtlePlus      //(1)
t.rectangle(3,3)         //(2)
t.forward(3)             //(3)
t.rectangle(3,3)         //(4)
t.rightTurn()           //(5)
t.rectangle(3,3)         //(6)
```

Discussion

The resulting diagram is sketched in Figure 3.3.

The default state for an object of type **TurtlePlus** is the same as that for **Turtle**, so after (1) *t* has marker state (0,0, “right(0)”) (with the pen “on” and an empty diagram).

Statement (2) adds the square ABCD to the diagram, and leaves the marker at (0,0), with direction “right(0)”.

Statement (3) moves the marker to (3,0), still facing “right(0)”. It goes over points already marked, so adds nothing new to the diagram.

Statement (4) then adds the square BEFC to the diagram and leaves the marker at (3,0) with direction “right(0)”.

Statement (5) turns the marker direction to “down(270)”.

Statement (6) adds the square BGHE to the diagram.

The marker finishes at (3,0) with direction “down(270)”.

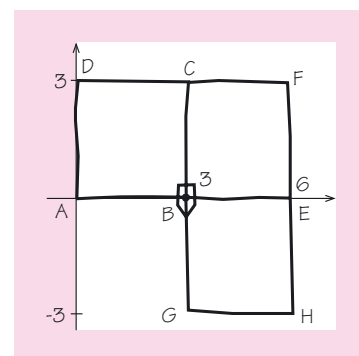


Figure 3.3

The code used to draw Figure 3.3 is not marker state invariant. In some contexts it is useful to ensure that code is marker state invariant, for example when the code is encapsulated in a function. It is easy to ensure that code is marker state invariant if desired, by use of *store* and *restore*.

We now look at the use of an object of type **TurtlePlus** to draw a desired diagram.

Activity 3.5

- Write a code fragment that will produce Figure 3.4 as the diagram of an object in **TurtlePlus**, and will leave the marker in the default state (as shown in Figure 3.4). (Make use of the *rectangle* method!)
- Write a code fragment that will produce Figure 3.5 as the diagram of an object in **TurtlePlus**.

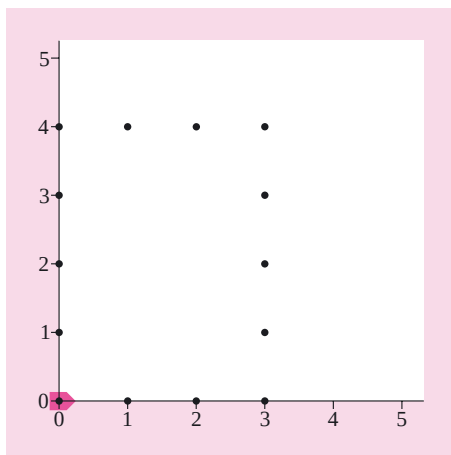


Figure 3.4

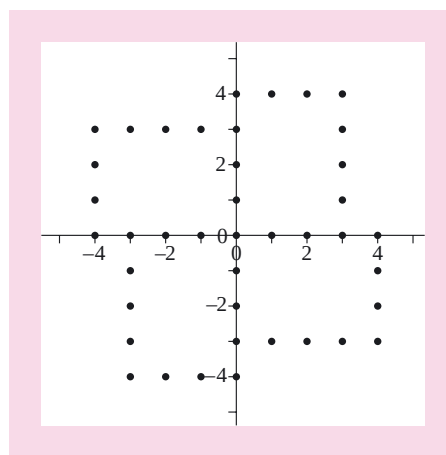


Figure 3.5

Discussion

- First declare a variable *t* of type **TurtlePlus**, then use the *rectangle* method once. After the **var** statement, *t* will be in the default state (0,0,“right(0)”). The first parameter of *rectangle* is the length of the first side drawn, which will be that from (0,0) to (3,0). That side has length 3. The second parameter is the length of the side from (3,0) to (3,4), which is 4. So the following code will draw Figure 3.4.

```
var t in TurtlePlus
t.rectangle(3,4)
```

- For Figure 3.5, we need to draw four rectangles. The upper-right rectangle is the same as Figure 3.4. To draw the upper-left rectangle, we must first rotate the marker anticlockwise, so that its direction is “up(90)”. A call of *rectangle*(3,4) will then draw a rectangle by first going up 3 places, then going left 4 places (and going on to complete the rectangle and return to (0,0,“up(90)”). Notice that this second rectangle requires the same parameter values as the first: (3,4). Another rotation anticlockwise, followed by another (3,4) rectangle will draw the lower-left

rectangle. Another left turn and (3, 4) rectangle will complete the pattern. Since we are repeating the same two steps four times, we can use a **for** loop, as below.

```
var t in TurtlePlus
var i in Int
for (i<--1 to 4)
{
  t.rectangle(3,4)
  t.leftTurn()
}
```

We can use a **for** loop in Activity 3.5(b) because the pattern in Figure 3.5 has “rotational symmetry”. Each of the other three rectangles is obtained by rotating the first rectangle, through 90, then 180, then 270 degrees. There are other code fragments that will draw this diagram, of course. The above is just a neat way of doing it. Short and neat code has its advantages over longer code fragments. It is easier to see its effect, and the repetition in the **for** loop reflects the symmetry of the diagram.

3.2 Diagonal lines

Use of *forward* draws a line that is either horizontal or vertical, parallel to either the x -axis or the y -axis. However, it is possible to move a turtle in a direction at 45 degrees to the current marker direction, by moving “one forward, turn left, one forward, turn right” and so on. The turtle then zigzags in a diagonal direction. While doing such a move, we can also mark some of the places visited by the turtle, to draw a diagonal line.

We now specify a function that adds a diagonal line to the diagram.

function *DIAGONAL*(t in Turtle, n in Int) **return** in Turtle

pre $n \geq 1$ and t has pen state “on”.

post The returned value is a turtle in the same state as t except for two things. A diagonal line has been added to the diagram. The diagonal line is drawn as follows. Turn 45 degrees anticlockwise from the current marker direction. Then add a diagonal line of $n + 1$ spots in that direction (including spots at the current marker position and the final marker position).

The marker position is changed to the final point on this diagonal line.

Activity 3.6

Sketch the diagram of the turtle t after the code fragment below has been executed.

```
var t in Turtle
t.leftTurn()
t.leftTurn()
t <-- DIAGONAL(t,10)
```

Discussion

The turtle t is created in the default state, then turned twice, so that the marker state is $(0, 0, \text{"left(180)"})$. A diagonal line of eleven spots is then drawn. This starts at $(0, 0)$, and is in the direction 45 degrees anticlockwise from “left(180)”, as sketched in Figure 3.6.

An implementation *impDiagonal* of *DIAGONAL* is given below.

```
function DIAGONAL(t,n)
{
    // impDiagonal
    var i in Int
    var temp in Turtle
    temp<--t.clone()
    for (i<--1 to n)
    {
        temp.forward(0)    // Mark current spot.
        temp.penOff()      // Take the pen off,
        temp.forward(1)    // move to next spot on diagonal,
        temp.leftTurn()
        temp.forward(1)
        temp.rightTurn()
        temp.penOn()       // and put the pen on again.
    }
    // The for loop marks n spots.
    temp.forward(0)       // Mark the last spot.
    return temp
}
```

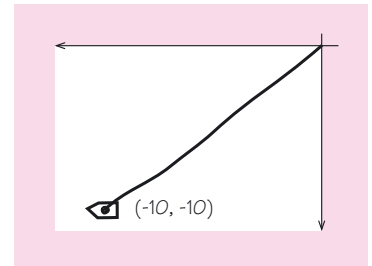


Figure 3.6

The natural specification of *DIAGONAL* with an input t that has the pen “off” would be that the marker would move without drawing a line. We could write code with that effect, but it would be more complicated. That is why we chose a precondition that the pen is “on”.

The ability to add diagonal lines to a diagram seems a good facility to add to the class **TurtlePlus**, thereby extending its drawing capabilities even further. Below we specify a further class **SuperTurtle** that **extends TurtlePlus**, and this time we add a method for plotting diagonal lines.

SuperTurtle

Class specification

SuperTurtle extends TurtlePlus

State, State notation and Default are as for **Turtle**.

Methods

diagonal(n in Int) state change

plus *rectangle* from **TurtlePlus**, and all the methods of **Turtle**.

method *diagonal*(n in Int) state change

pre $n \geq 1$ and the receiver has pen state “on”.

post The state of the receiver is changed in two ways.

A diagonal line is added to the diagram. The diagonal line is drawn as follows. Turn 45 degrees anticlockwise from the current marker direction. Then add a diagonal line of $n + 1$ spots in that direction (including spots at the current marker position and the final marker position).

The marker position is changed to the final point on this diagonal line.

To implement **SuperTurtle**, we need to modify the code for the function *DIAGONAL* so as to obtain an implementation of the method.

Activity 3.7

Give an implementation of the class **SuperTurtle** in a form suitable for the WorkPad.

Discussion The necessary code is given below. We need to declare the class and link the method to its implementation. Then we give the implementation of the method *diagonal*. In this, we change the code for *impDiagonal* in the following ways.

1. “method” replaces “function”.
2. No input turtle needed. The method is to be applied to the receiver object. Hence *temp* is replaced by *this* throughout.
3. There is no value returned; the method changes the state of the receiver.

```

Class SuperTurtle
{
    this.diagonal calls diagonalSuperTurtle
}

SuperTurtle extends TurtlePlus

method diagonalSuperTurtle(n)
{
    var i in Int
    for (i<--1 to n)
    {
        this.forward(0)
        this.penOff()
        this.forward(1)
        this.leftTurn()
        this.forward(1)
        this.rightTurn()
        this.penOn()
    }
    this.forward(0) // The for loop marks the first n dots
                  // the last dot
}

```

In this section we have added two extra drawing features to turtles, and have enhanced the class **Turtle** first to **TurtlePlus** then to **SuperTurtle**, by adding these new features as methods, using **extends**. A turtle object that is an instance of **SuperTurtle** can draw any of: a single dot, a line, a rectangle or a diagonal line.

Both **TurtlePlus** and **SuperTurtle** are available in the WorkPad as library classes.

Objectives for Section 3

After studying this section you should be able to do the following.

- ☐ Understand the specifications of the functions *RECTANGLE* and *DIAGONAL*.
- ☐ Understand the specifications of the classes **TurtlePlus** and **SuperTurtle**.
- ☐ Understand the implementations of the classes **TurtlePlus** and **SuperTurtle**.
- ☐ Sketch the diagrams produced by code fragments involving the methods *rectangle* and *diagonal*.
- ☐ Given a diagram, produce a code fragment that uses either **TurtlePlus** or **SuperTurtle** to draw it.

Exercises on Section 3

Exercise 3.1

Draw the diagram and give the final marker state after the code fragment below has been executed.

```
var t in SuperTurtle
t.forward(6)
t.leftTurn()
t.forward(6)
t.leftTurn()
t.diagonal(6)
```

Exercise 3.2

Draw the diagram and give the final marker state after the code fragment below has been executed.

```
var t in TurtlePlus
var i in Int
for (i<--1 to 4)
{
  t.rectangle(5-i,1)
  t.leftTurn()
  t.forward(2)
  t.rightTurn()
}
```

Exercise 3.3

Give a code fragment that uses an object of type **SuperTurtle** to produce the diagram in Figure 3.7. Ensure that the final marker state is (0,0,"right(0)").

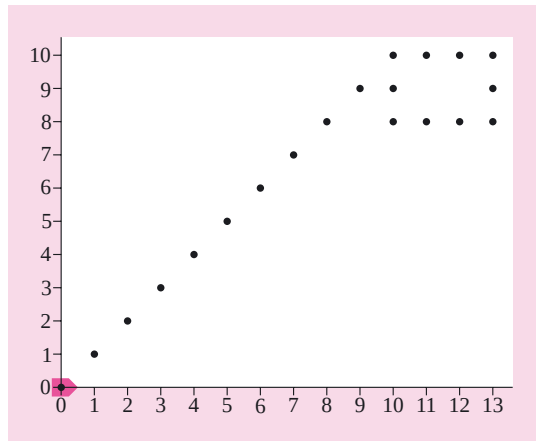


Figure 3.7

4 Turtle fractals

In this section, we shall see how the class **Turtle** is capable (in principle, at least) of drawing diagrams whose structure is quite complicated and subtle. To do this, we can use recursion. However, this is **not** an idea that we expect you to become familiar with now. It is discussed later in the course.

To start, consider the code fragment below, which draws the diagram in Figure 4.1.

```
var t in Turtle
t.forward(16)
t.store()
t.leftTurn()
t.forward(8)
t.restore()
t.forward(16)
```

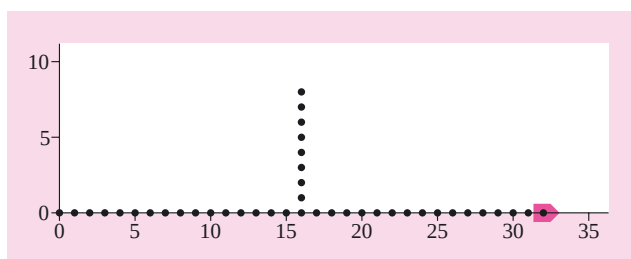


Figure 4.1

Read Only

The material in this section is not assessed directly.

When this fragment is executed, a line from (0,0) to (16,0) is drawn and then the marker state (16,0,“right(0)”) is stored. Next, the turtle is turned to face up, and a line from (16,0) to (16,8) is drawn. The turtle’s state is then restored to (16,0,“right(0)”), and the line from (16,0) to (32,0) is drawn.

The diagram in Figure 4.1 consists of a line of length 32 with a centrally placed “spike” of length 8 at right angles to it. The marker ends up in the same position, and with the same direction, that would have been produced by the statement *t.forward(32)*. The diagram is drawn as three lines, two of length 16 (which is half of 32), with a line of length 8 (which is a quarter of 32) at right angles to these.

We want now to consider a function that generalises the idea used to draw this diagram. A specification and implementation are given below. The function draws a figure where the length 32 is generalised. The function has two inputs: a turtle and a length (with identifier *side*). To produce symmetric diagrams, we impose a precondition that *side* is a multiple of 4.

function *SPIKE1*(*t* in Turtle, *side* in Int) **return** in Turtle

pre *side* is a multiple of 4.

post The returned turtle is formed from *t* by changing two features.

A “spike line”, as illustrated in Figure 4.1, is added to the diagram of *t* (starting at the marker position of *t* and in the marker direction of *t*). This has length *side*, and the “spike” has length *side*/4.

The marker is moved from the marker position of *t* by a distance *side*, in the marker direction of *t*.

```
function SPIKE1(t,side)
{
  var temp in Turtle
  temp<-- t.clone()
  temp.forward(DIV(side,2))
  temp.store()
  temp.leftTurn()
  temp.forward(DIV(side,4))
  temp.restore()
  temp.forward(DIV(side,2))
  return temp
}
```

So the following code fragment will draw the diagram in Figure 4.1.

```
var t in Turtle
t<-- SPIKE1(t,32)
```

We shall call a diagram such as that in Figure 4.1 a “spike line” (in contrast to the “straight line” produced by *t.forward(side)*). This spike line is made up of three straight lines. Two of these combine to form a single line whose length is the input integer *side*, with a spike half way along this and at right angles to it, whose length is a quarter of *side*.

Now we need to take a leap in imagination. Suppose that, in the spike line, we replace each of the three straight lines by a (smaller) spike line. Perhaps you can picture the result? It is relatively easy to give code that implements a function *SPIKE2* that does this. It is not easy to specify the

postcondition of this function, and we shall not attempt to do so. We will impose a precondition that *side* is a multiple of 16, because we want *side*/4 to be a multiple of 4 (so that it satisfies the precondition of *SPIKE1*). To obtain the code for *SPIKE2*, we take the implementation of *SPIKE1* and replace each call of *forward* with an assignment to the corresponding value of *SPIKE1* itself.

```
function SPIKE2(t,side)
{
    // pre: side is a multiple of 16.
    var temp in Turtle
    temp<--t.clone()
    temp<--SPIKE1(temp,DIV(side,2))  //(1)
    temp.store()
    temp.leftTurn()
    temp<--SPIKE1(temp,DIV(side,4))
    temp.restore()
    temp<--SPIKE1(temp,DIV(side,2))
    return temp
}
```

For instance, in statement (1) above, the equivalent statement in the implementation of *SPIKE1*, which was *temp.forward(DIV(side, 2))*, has been replaced by the statement *temp ← SPIKE1(temp, DIV(side, 2))*. Instead of adding a straight line to the diagram of the turtle *temp*, this statement adds a spike line.

The effect of *SPIKE2* is illustrated in Figure 4.2, which shows the diagram produced by the following code fragment. To see how this diagram arises, we can look at the diagram for *SPIKE1* in Figure 4.1, and replace each straight line in it by a spike line of the appropriate length.

```
var t in Turtle
t <-- SPIKE2(t,32)
```

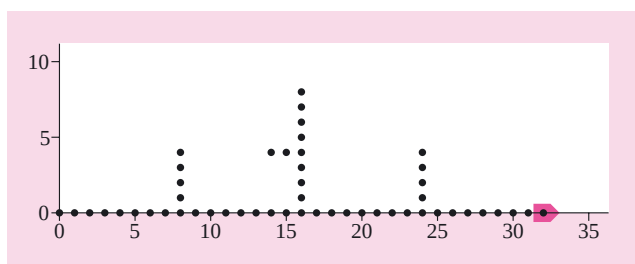


Figure 4.2

After execution of *t ← SPIKE2(t, 32)*, the state of the marker in *t* is the same as it is after execution of *t ← SPIKE1(t, 32)*, or after execution of *t.forward(32)*. The diagram for *SPIKE2* is different, though. For lack of a better way of expressing it, we end up with an “even spikier” line than that of *SPIKE1*.

In Figure 4.2, we can see that the diagram for *SPIKE2* (a “spike2 line”) is made up of three “spike1 lines”. Now if we replace each of the three spike1 lines in Figure 4.2 by a spike2 line, we will obtain the diagram for the next

member of the “spike” family of functions. An implementation of this is given below.

```
function SPIKE3(t,side)
{
    // pre: side is a multiple of 64
    var temp in Turtle
    temp<--t.clone()
    temp<--SPIKE2(temp,DIV(side,2))
    temp.store()
    temp.leftTurn()
    temp<--SPIKE2(temp,DIV(side,4))
    temp.restore()
    temp<--SPIKE2(temp,DIV(side,2))
    return temp
}
```

The effect of *SPIKE3* is illustrated in Figure 4.3, which shows the diagram produced by the following code fragment.

```
var t in Turtle
t <-- SPIKE3(t,64)
```

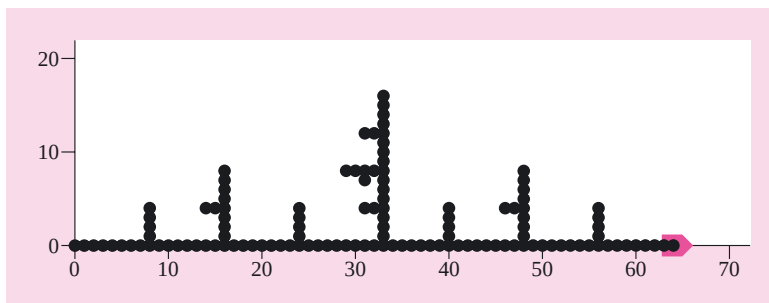


Figure 4.3

The procedure of forming each new “spike” function from the preceding one can be repeated indefinitely, at least in principle. This would produce a “family” of functions, where *SPIKE n* calls *SPIKE $n - 1$* , and so on, until we get to *SPIKE1*. (In practice, the resolution of the screen will prevent us from showing diagrams for all but a few of these functions.)

The diagram for each member of this family of functions is produced from three copies of the diagram of the previous member of the family. We say that the diagrams are produced by “self-similarity”. (If we imagine continuing this process an indefinitely large number of times, the resulting diagram is called a **self-similar fractal**.)

We can extend the family of spike functions “backwards”. We will use *forward(side)* to implement a function that starts the family off, which we will call *SPIKE0*.

```
function SPIKE0(t,side)
{
  var temp in Turtle
  temp<--t.clone()
  temp<--temp.forward(side)
  return temp
}
```

In the implementation of *SPIKE1* we can then replace each of the three statements of the form *temp.forward(n)* by *temp ← SPIKE0(temp, n)*.

Now if we want to use the WorkPad to produce the diagram for *SPIKE7*, say, we will need to have in the function window (in the correct order) the implementations of all the functions: *SPIKE0*, *SPIKE1*, *SPIKE2*, *SPIKE3*, *SPIKE4*, *SPIKE5*, *SPIKE6*, and *SPIKE7*. Such an approach is tiresome, unwieldy, and likely to be prone to errors. Also, it does not fully exploit through reuse the similarity of the code in all these implementations. To do this, we can use recursion. We indicate below how this can be achieved.

You may notice two things here. Firstly, we are no longer specifying a family of different functions. Instead, the *n* in *SPIKE_n* has become an additional input to a single function. Second, the implementation of this function calls on itself! This looks self defeating. Two key points enables it to work. First, we use a call to *SPIKE* with a smaller value of the input *n*. Second, we must supply the code for the case where *n* is 0.

```
function SPIKE(n,t,side)
{ // pre: side is a multiple of 4 to the power n.
  var temp in Turtle
  temp<--t.clone()
  if (n == 0) then
  {
    temp.forward(side)
  }
  else
  {
    temp<--SPIKE(n-1,temp,DIV(side,2))
    temp.store()
    temp.leftTurn()
    temp<--SPIKE(n-1,temp,DIV(side,4))
    temp.restore()
    temp<--SPIKE(n-1,temp,DIV(side,2))
  }
  return temp
}
```

Summary of Unit 5

1. The state of an M263 turtle is determined by four features (the marker, the pen, the diagram and the markerStack). The marker state has a position, given by a pair of integers showing the coordinates of the marker, and a direction, which is one of four possible directions pointing along the axes, and given as “right(0)”, “up(90)”, “left(180)” or “down(270)”. The pen state is either “on” or “off”. The diagram is a set of spots, showing the points on the integer grid that have been marked. The markerStack is a stack of marker states, and can be used to store the current marker state, and later to restore the marker to that state.
2. The implementation of the class **Turtle** has four attributes: *attMarker*, which is of type **Marker**, *pen*, which is of type **Bool**, *markerStack*, which is of type **Stack of Marker**, and *diagramStack*, which is of type **Stack of Spot**. (In abstract terms, the diagram is a set of spots, but **Stack** is the only type available so far that can be used to implement a collection whose size varies.) The class **Marker** was introduced to implement the marker attribute of a turtle. It is specified to be an extension of the class **Spot**, and so has access to the methods and attributes of **Spot**. We do not want **Turtle** to have access to all the methods of **Marker** (and so of **Spot**), so **Turtle** uses **Marker** in composition, and is not an extension of **Marker**.
3. We introduced classes **TurtlePlus** and **SuperTurtle**, that add to the functionality of the class **Turtle** in certain ways. **TurtlePlus** extends **Turtle**, and has an additional method that adds a rectangle to the diagram. **SuperTurtle** extends **TurtlePlus**, and has another additional method that adds to the diagram a line at 45 degrees to the axes.

Solutions to the exercises

Solution 1.1

The first code fragment leaves the marker in the default state, $(0,0,\text{"right}(0)\text{"})$. This state is stored by statement (1) in the second fragment. A line of spots from $(0,0)$ to $(10,0)$ is then added to the diagram by statement (2). (Some of these were already marked by the first fragment, but this is of no consequence.) Then the marker is turned to point $\text{"down}(270)\text{"}$. Statement (4) then adds a line of spots from $(10,0)$ to $(10,-5)$. Statement (5) then returns the marker state to that stored by statement (1), which was the default state. So the final marker state is $(0,0,\text{"right}(0)\text{"})$. The resulting diagram and marker state are shown in Figure S1. (Remember that the square has already been drawn by the first fragment.)

Solution 1.2

(a) We can use the following strategy.

Turn to point $\text{"up}(90)\text{"}$.

Store the state $(0,0,\text{"up}(90)\text{"})$.

Take the pen off.

Move to $(0,5)$.

Put the pen on.

Draw the line from $(0,5)$ to $(0,10)$.

Restore the marker to the state $(0,0,\text{"up}(90)\text{"})$.

This strategy leads to the following code.

```
t.leftTurn()
t.store()
t.penOff()
t.forward(5)
t.penOn()
t.forward(5)
t.restore()
```

(b) The first iteration of *Fragment1* in the **for** loop will draw the diagram shown in Figure 1.17, and leave the marker at $(0,0,\text{"up}(90)\text{"})$. The second iteration of *Fragment1* in the **for** loop will first turn the marker to point $\text{"left}(180)\text{"}$, then move along the negative x -axis and mark the points between $(-5,0)$ and $(-10,0)$, and restore the marker state to $(0,0,\text{"left}(180)\text{"})$. The third iteration of *Fragment1* in the **for** loop will turn the marker to point $\text{"down}(270)\text{"}$, then move the marker along the negative y -axis and mark the points between $(0,-5)$ and $(0,-10)$, and restores the marker to $(0,0,\text{"down}(270)\text{"})$. The diagram and marker in the final state of the turtle are shown in Figure S2.

To describe the final state of the turtle more fully, we should also note that the final markerStack will be empty. *Fragment1* stores one marker state in the markerStack, then the *restore* statement deletes that stored marker. Thus *Fragment1* leaves the markerStack empty, and so repeating *Fragment1* three times also leaves the markerStack empty. (Note that *Fragment1* is not marker state invariant. Repeating a marker state invariant code fragment in a **for** loop would just draw the same diagram over and over again.)

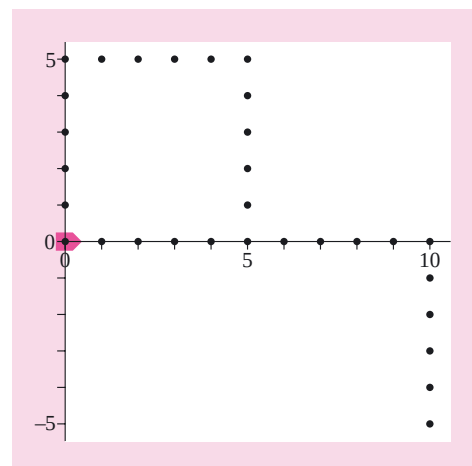


Figure S1

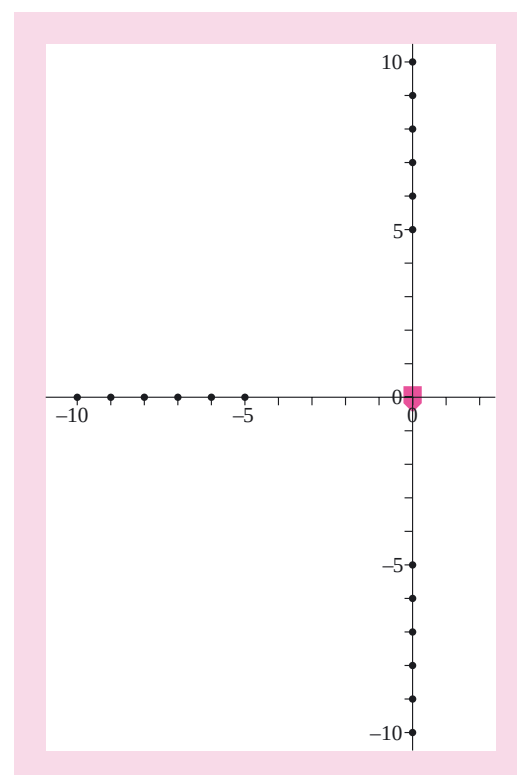


Figure S2

Solution 2.1

(a) Recall that the pen attribute of **Turtle** has type **Bool**, and the pen being “on” is represented as **true**. So we just need the following statement.

```
this.pen <-- true
```

(b) The following statements are needed.

```
this.penOff calls penOffTurtle
```

```
method penOffTurtle()
{
  this.pen <-- false
}
```

Solution 2.2

We want an object of the new class to have all the features of a turtle — all its structure, and access to all the methods of the class **Turtle** — together with additional functionality (the ability to erase a spot from the diagram). This indicates that extension is likely to be appropriate. An object of the new class will “be” a turtle (with access to an additional *erase* method).

Solution 2.3

The class **HLine** can be implemented in the WorkPad as below.

```
Class HLine
{
  this.attTurtle in Turtle      //(1)
  this.moveTo calls moveToHLine
  this.draw calls drawHLine
}

HLine extends Base
```

Statement (1) ensures that **HLine** will inherit its default state from **Turtle**.

```
method moveToHLine(x,y)
{
  this.attTurtle.attMarker.xPos <-- x
  this.attTurtle.attMarker.yPos <-- y
}

method drawHLine(n)
{
  this.attTurtle.store()
  this.attTurtle.forward(n)
  this.attTurtle.restore()
}
```


Solution 3.1

The diagram is shown in Figure S3. The final marker state is $(0, 0, \text{"left(180)"})$.

Solution 3.2

The diagram is shown in Figure S4. The final marker state is $(0, 8, \text{"right(0)"})$. The diagram is drawn as follows.

The turtle starts at $(0, 0, \text{"right(0)"})$.

The first execution of the **for** loop (with $i = 1$) calls `rectangle(4, 1)`, which marks the bottom 10 spots of the diagram. The marker is then moved to $(0, 2, \text{"right(0)"})$.

The second execution of the **for** loop (with $i = 2$) calls `rectangle(3, 1)`, which marks the next 8 spots in the diagram. The marker is then moved to $(0, 4, \text{"right(0)"})$.

The third execution of the **for** loop (with $i = 3$) calls `rectangle(2, 1)`, which marks the next 6 spots of the diagram. The marker is then moved to $(0, 6, \text{"right(0)"})$.

The fourth (and last) execution of the **for** loop (with $i = 4$) calls `rectangle(1, 1)`, which marks the next 4 spots of the diagram. The marker is then moved to $(0, 8, \text{"right(0)"})$, and the spot at $(0, 8)$ is marked.

Solution 3.3

We can draw this diagram using the following strategy.

Draw the diagonal line from $(0, 0)$ to $(10, 10)$. Move to $(10, 8)$ with direction `"right(0)"`. Draw the rectangle.

This will certainly not leave the marker at $(0, 0)$. The easiest way to leave the marker state as required is to *store* the default marker state at the start and *restore* it at the end. Suitable code is given below.

```
var t in SuperTurtle
t.store()
t.diagonal(10)
t.rightTurn()
t.forward(2)
t.leftTurn()
t.rectangle(3, 2)
t.restore()
```

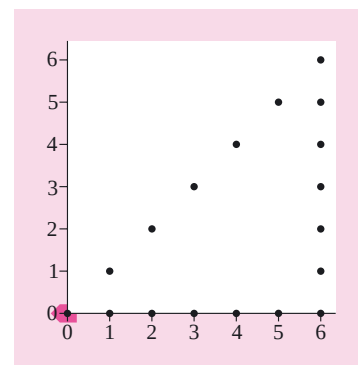


Figure S3

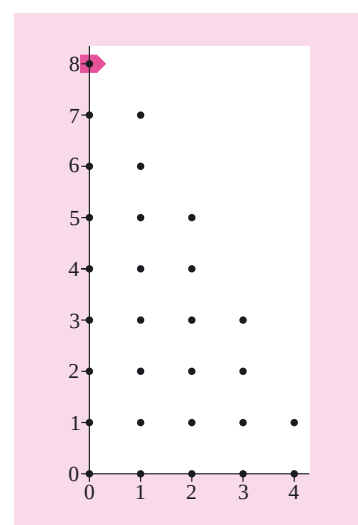


Figure S4

Index

composition of classes, 24

diagram, 6

get method, 30

hide, 23

Marker, 21

marker, 5

marker state, 7

marker state invariant, 16

markerStack, 7

pen, 6

set method, 30

SuperTurtle, 37

Turtle, 5

TurtlePlus, 33

WorkPad class library, 20